Transcoding Text Views

Range adapters for character encoding conversion

Document #:	D4246R1
Date:	2026-11-12
Project:	Programming Language C++
Audience:	SG16 SG9 LEWGI LEWG

Reply-to:	Steve Downey <sdowney2@bloomberg.net>

Contents

0.1 Abstract . . . 2
0.2 Changes since R0 . . . 2
0.3 Context and Motivation . . . 3
0.4 Comparison Table . . . 4
0.5 Design Rationale . . . 7
0.6 Design . . . 9
0.7 Design Decisions . . . 18
0.8 Methods . . . 21
0.9 Testing . . . 21
0.10 Impact on the Standard . . . 22
0.11 Open Questions for SG16 . . . 23
0.12 Summary of the Design . . . 23
0.13 Wording . . . 24
Header `<transcode>` synopsis [transcode.syn] . . . 24
Error types [transcode.errors] . . . 33
Range requirements [transcode.reqs] . . . 35
Codec requirements [transcode.codec.reqs] . . . 35
Class template `decode_view` [transcode.codec.decode] . . . 37
Class template `encode_view` [transcode.codec.encode] . . . 38
Transcoding pipelines [transcode.pipeline] . . . 40
Encodings [transcode.whatwg.encoding] . . . 40
Decoders [transcode.whatwg.decoders] . . . 42
Encoders [transcode.whatwg.encoders] . . . 43
Codecs for an encoding chosen at run time [transcode.whatwg.dynamic] . . . 43
Decoding views [transcode.whatwg.decode] . . . 44
Encoding views [transcode.whatwg.encode] . . . 44
Eager transcoding [transcode.string] . . . 45
Eager bulk conversion [transcode.bulk] . . . 45
iconv adaptors [transcode.iconv] . . . 47
Header `<null_term>` synopsis [null.term.syn] . . . 51
Class `null_sentinel_t` [null.term.sentinel] . . . 51
Class template `null_term_view` [null.term.view] . . . 52
`views::null_term` [null.term.adaptor] . . . 52
0.14 Acknowledgements . . . 52
0.15 Prior Art . . . 52
0.16 Relationship to Other Proposals . . . 53
0.17 References . . . 55

0.1 Abstract

"It seems like a really messy situation and you're proposing to inherit that mess into the C++ standard."
— Jan Schultke [Schultke]

He is right about the mess. The encodings are underspecified, the implementations disagree about the edge cases, and no amount of standards work will make Shift_JIS pleasant. However, the data exists, and C++ programs have to read it. The alternative to a specified transcoder is the world we have, where every program invents its own recovery rules and gets different answers from the same bytes. WHATWG has already done the work of pinning down what every byte means. What this proposal inherits is that work.

We propose a set of transcoding facilities for text between character encodings, centered on separate WHATWG decode and encode adaptors built over codec objects that a program can also supply, eager bulk decode and encode helpers, a convenience composed transcoder, and an `iconv`-based adaptor for broader encoding support.

The views are lazy and compose in pipelines: decode byte sequences to Unicode scalar values, encode Unicode back to bytes. The eager bulk operations collect into containers or write through output iterators, for when a concrete result is more convenient than a view pipeline.

Codec semantics follow the WHATWG Encoding Standard [WHATWG-Encoding]: the same byte-to-scalar mappings and the same error recovery the browsers implement. For encodings not covered by WHATWG, an `iconv`-based adaptor provides access to the platform's native transcoding capabilities where the implementation supplies that facility. The reference implementation currently requires `iconv` at package-configuration time, even though its WHATWG implementation is otherwise portable header-only source.

The wording in this revision is generated from the reference implementation's headers, by a tool that reads the shipping declarations and renders them as draft clauses. It is not a transcription, and it is not maintained alongside the code: a header that changes without the wording changing with it fails the implementation's CI. The check is a `make` target, so a reviewer need not take the claim on faith.

0.2 Changes since R0

This revision responds to SG16's review of P4246R0. The design changed, and the wording is regenerated from the reference implementation as it now stands.

0.2.1 Design

- The WHATWG vocabulary and adaptors moved into `whatwg::` `whatwg::encoding` (was `codec`), `whatwg::error`, `whatwg::get_encoding`, `whatwg::bom_sniff`, `whatwg::decode<C>`, `whatwg::encode<C>`, `whatwg::transcode<From, To>`. The generic codec concepts were renamed `text_decoder` and `text_encoder`, with `flushable_` and `single_byte_` refinements, and stay at the top level. The `whatwg` Namespace explains the split and flags it as open.
- `transcode_error_kind::{replacement, expected}` is now `error_mode::{replacement, report}`.
- The WHATWG codecs are available as codec objects, `whatwg::decoder<C>` and, where the Encoding Standard defines an encoder, `whatwg::encoder<C>`. The WHATWG adaptors are specified as the codec-object adaptors over them, so `whatwg::decode<C>` is `decode(whatwg::decoder<C>{})`, and there is one decode view and one encode view. `transcode(from, to)` composes any decoder with any encoder, so built-in and program-defined codecs mix.
- New concepts and protocol pieces: `flushable_text_encoder`, `bom_aware_text_decoder`, `single_byte_text_encoder`, `decode_result::is_empty`, and encoder replacement bytes. An encoder's replacement bytes also apply to what `flush()` returns: in replacement mode the view yields them, or `'?'` if there are none, and in report mode the error is a single element. The codec-object views model `borrowed_range` when their base does. See Decoder and Encoder Concepts.
- The bridge to `std::text_encoding::id` goes in both directions, `whatwg::get_encoding(text_encoding::id)` and `whatwg::to_text_encoding_id`, and Why Not `text_encoding::id`? says why it is a bridge and not the interface.
- The Design section opens with the three layers, WHATWG, codec objects and system `iconv`, and a worked example of a codec the library does not ship.
- The cost of runtime selection is measured, and Template Parameter vs Runtime Selection reports it.
- `whatwg::encoder<x_user_defined>` exists: the Encoding Standard defines an x-user-defined encoder, and R0 said it did not. Only `utf_16be`, `utf_16le` and `replacement` have no encoder.

— Runtime selection is shown through the library's own entry points, which take labels and do the dispatch, rather than through a hand-written `switch`.
— Why Not `text_encoding::id`? quotes the Encoding Standard on why its label matching is stricter than UTS #22, and says why no cast between the enumerations can be correct.
— The bulk-conversion section shows writing into a caller-owned buffer, since the views allocate nothing.
— An encoding chosen at run time no longer needs a hand-written `switch` to reach code points. `whatwg::dynamic_decoder` and `whatwg::dynamic_encoder` are codec objects for it, and the bulk helpers take the encoding as a value or a label. Template Parameter vs Runtime Selection says what each costs.
— `decode_to_or_error` and `encode_to_or_error` collect a whole text or return the first error, in WHATWG and codec-object forms. They are the eager counterparts of the `_or_error` views, as `iconv_transcode_to_or_error` already was for the `iconv` adaptor.
— The Codec Protocol Is a Coalgebra names the shape the codec concepts have, and derives from it the decisions the rest of the rationale argues one at a time.
— The namespace and `single_byte_text_encoder` are open questions for SG16.

0.2.2 Wording

— The enumerator-value claim is corrected, and the rationale is removed from [transcode.whatwg.encoding].
— Remarks that had lost their clauses are routed to the clauses they belong to.
— *Mandates* and *Constraints* replace "is required to be".
— The clause headings carry stable names, and an editor's note covers the nested namespaces.
— `whatwg::decoder<C>::flush` is constrained on the exposition-only *`has-decode-state`*`<C>`, where the wording had shown a comparison of `detail::` types.
— The normative reference in [intro.refs] names the Encoding Standard's June 2025 Review Draft, a dated document, where R0 named the living standard.
— The synopsis no longer names the adaptor closure types. `decode`, `decode_or_error`, `encode`, `encode_or_error` and `transcode(from, to)` are declared `constexpr auto`, the way `std::ranges::to` is in [range.utility.conv], and their clauses say what they return.

0.2.3 Other corrections

— [transcode.iconv] conditions its facilities on an available POSIX-conforming `iconv` interface, which the operating system or a third-party library may supply, and `iconv_t` is that interface's type. R0 conditioned them on "an implementation that supplies an iconv conversion facility" and left `iconv_t` implementation-defined.
— `whatwg::bom_sniff` and the `null_term` sentinel's comparison state their constraints as *Constraints:* paragraphs rather than as requires-clauses in the declarations.
— The paragraph on `<codecvt>` names the removed facets as what allocated and dispatched virtually, where R0 left the subject to be inferred.
— Three passages state what is gained and given up directly: the rationale for the template-parameter codec, the named bulk helpers, and the open question for SG16.
— Prior Art opens on the libraries it draws from.

0.3 Context and Motivation

IANA registered names for character sets before anyone had specified what the names meant. The registry says a document is "Shift_JIS"; it does not say what `0x81 0x40` decodes to. C answered with the multibyte functions and POSIX standardized `iconv`, and both work, as long as you do not care which implementation you get. glibc, musl, and the BSD and macOS libraries disagree about error recovery and about which encodings exist at all, and all of them depend on how the system was configured. C++ added `std::codecvt` in `<locale>`, and the `<codecvt>` header's Unicode conversion facets on top of it. `<codecvt>` — `codecvt_utf8`, `codecvt_utf16`, `codecvt_utf8_utf16` — was deprecated in C++17 and removed in C++26 [P2871R3], and the `char16_t` and `char32_t` facet specializations deprecated in C++20 went with it [P2873R3]. `std::codecvt` itself is still in `<locale>`; what was removed is everything in the pair that converted Unicode. The removed facets allocated on every call and dispatched through a virtual interface to do so; in the accompanying benchmark, the C89 `mbstowcs` is faster.

The WHATWG Encoding Standard specified the missing part, for exactly the encodings the web still carries. It gives the byte-to-scalar mapping for every byte value, says what each malformed sequence produces, and

the Web Platform Tests [WPT-Encoding] pin all of it across the browser engines. C++ has since added `std::text_encoding`, which names an encoding. Nothing in the standard converts one, and "Why Not `text_encoding::id`?" says why this proposal does not start from it.

Everyone should be using UTF-8. Postel's law applies anyway: conservative in what we emit, liberal in what we accept. There is a great deal of existing data in legacy encodings that backend C++ systems have to read, web interchange has not finished migrating, and email still generates and routes large volumes of non-UTF-8 text.

Real data is also badly formed. Emails and web pages lie about their declared encodings, and concatenate mismatched encodings into a single payload. So the facility has to keep going past a bad byte and let the caller decide what that meant. Aborting on the first bad byte is not an option when the bytes are what you were sent.

0.4 Comparison Table

0.4.1 Decoding UTF-8 to Unicode code points

Before: manual loop

```
std::vector<char32_t> decode_utf8(
    std::string_view input) {
  std::vector<char32_t> result;
  size_t i = 0;
  while (i < input.size()) {
    unsigned char b = input[i];
    char32_t cp; int extra;
    if (b < 0x80) { cp = b; extra = 0; }
    else if ((b & 0xE0) == 0xC0)
      { cp = b & 0x1F; extra = 1; }
    else if ((b & 0xF0) == 0xE0)
      { cp = b & 0x0F; extra = 2; }
    else if ((b & 0xF8) == 0xF0)
      { cp = b & 0x07; extra = 3; }
    else { result.push_back(U'\xFFFD');
           ++i; continue; }
    if (i + extra >= input.size()) {
      result.push_back(U'\xFFFD'); break;
    }
    for (int j = 0; j < extra; ++j) {
      unsigned char c = input[++i];
      if ((c & 0xC0) != 0x80) {
        cp = U'\xFFFD'; break;
      }
      cp = (cp << 6) | (c & 0x3F);
    }
    // Missing: overlong, surrogate checks
    result.push_back(cp);
    ++i;
  }
  return result;
}
```

After: `whatwg::decode`

```
std::vector<char32_t> decode_utf8(
    std::string_view input) {
  return input
    | whatwg::decode<whatwg::encoding::utf_8>
    | std::ranges::to<std::vector>();
}
```

0.4.2 Transcoding Shift_JIS to UTF-8

Before: `iconv`

```
std::string shift_jis_to_utf8(
    std::string_view input) {
  iconv_t cd = iconv_open("UTF-8",
                          "SHIFT_JIS");
  if (cd == (iconv_t)-1)
    throw std::runtime_error("iconv_open");

  std::string result;
  result.resize(input.size() * 4);

  char* inbuf = const_cast<char*>(
                  input.data());
  size_t inleft = input.size();
  char* outbuf = result.data();
  size_t outleft = result.size();

  while (inleft > 0) {
    size_t r = iconv(cd, &inbuf, &inleft,
                     &outbuf, &outleft);
    if (r == (size_t)-1) {
      if (errno == E2BIG) {
        size_t used = outbuf - result.data();
        result.resize(result.size() * 2);
        outbuf = result.data() + used;
        outleft = result.size() - used;
      } else if (errno == EILSEQ) {
        // Skip invalid byte
        ++inbuf; --inleft;
        // Append replacement char
        *outbuf++ = '\xEF';
        *outbuf++ = '\xBF';
        *outbuf++ = '\xBD';
        outleft -= 3;
      } else {
        iconv_close(cd);
        throw std::runtime_error("iconv");
      }
    }
  }
  result.resize(outbuf - result.data());
  iconv_close(cd);
  return result;
}
```

After: decode then encode

```
std::string shift_jis_to_utf8(
    std::string_view input) {
  return input
    | whatwg::decode<whatwg::encoding::shift_jis>
    | whatwg::encode<whatwg::encoding::utf_8>
    | std::ranges::to<std::string>();
}
```

0.4.3 Converting a null-terminated C string

Before: `mbstowcs`

```
std::wstring from_cstring(const char* s) {
  // Assumes locale is set correctly
  std::setlocale(LC_ALL, "");
  size_t len = std::mbstowcs(nullptr, s, 0);
  if (len == (size_t)-1)
    throw std::runtime_error("mbstowcs");
  std::wstring result(len, L'\0');
  std::mbstowcs(result.data(), s, len + 1);
  return result;
  // Problems:
  // - Global locale state
  // - wchar_t is not portable
  // - No error recovery
  // - Two passes required
}
```

After: `views::null_term`

```
auto from_cstring(const char* s) {
  return views::null_term(s)
    | whatwg::decode<whatwg::encoding::utf_8>;
  // Returns lazy view of char32_t
  // No global state
  // Portable Unicode scalars
  // Single pass, errors yield U+FFFD
}
```

0.4.4 Processing with error visibility

Before: manual tracking

```
// No standard way to detect errors
// during transcoding. Either:
// 1. Errors are silently replaced
// 2. Exceptions thrown mid-stream
// 3. Custom state machine required

bool has_errors = false;
std::vector<char32_t> result;
// ... complex manual decoding with
// error tracking interspersed ...
if (has_errors) {
  log_warning("Invalid UTF-8 detected");
}
```

After: `_or_error`

```
for (auto r : input
    | whatwg::decode_or_error<whatwg::encoding::utf_8>)
  if (r.has_value()) {
    process(*r);
  } else {
    log_warning(r.error());
    process(U'\xFFFD');
  }
}
```

0.4.5 Bulk conversion to owned storage

Before: manual collection loop

```
std::vector<char32_t> decode_utf8(
    std::string_view input) {
  std::vector<char32_t> result;
  for (char32_t cp : input
      | whatwg::decode<whatwg::encoding::utf_8>) {
    result.push_back(cp);
  }
  return result;
}
```

After: `decode_to`

```
std::vector<char32_t> decode_utf8(
    std::string_view input) {
  return whatwg::decode_to<whatwg::encoding::utf_8>(inpu
}
```

We propose `decode_to` / `encode_to` / `decode_into` / `encode_into`, and they add nothing the views do not already do. The views satisfy the range concepts that `std::ranges::to` and `std::ranges::copy` require, so the standard algorithms already do the job, and there is no measurable difference in speed between a named `decode_to` and the equivalent `view | ranges::to` composition.

The names are worth having anyway. Converting a buffer to a container is the operation users come for, and `whatwg::decode_to<whatwg::encoding::utf_8>(input)` is what they look for first. Withholding the name only moves the question to every code review and every committee discussion of this facility. Each helper is a one-line forward to the pipeline it names; what the proposal adds is four names and the wording that specifies them.

The named helpers and pipelines are the same operations:

```
whatwg::decode_to<C>(r)          is  r | whatwg::decode<C> | ranges::to<vector<char32_t>>()
whatwg::encode_to<C, Cont>(r)    is  r | whatwg::encode<C> | ranges::to<Cont>()
whatwg::decode_into<C>(r, out)   is  ranges::copy(r | whatwg::decode<C>, out)
whatwg::encode_into<C>(r, out)   is  ranges::copy(r | whatwg::encode<C>, out)
```

Two more answer the question the `_or_error` views leave with the caller: the whole text, or the first thing wrong with it.

```
decode_to_or_error(r, codec)             ->  expected<vector<char32_t>, whatwg::error>
encode_to_or_error<Cont>(r, codec)       ->  expected<Cont, whatwg::error>
whatwg::decode_to_or_error<C>(r)         is  decode_to_or_error(r, whatwg::decoder<C>{})
whatwg::encode_to_or_error<C, Cont>(r)   is  encode_to_or_error<Cont>(r, whatwg::encoder<C>{})
```

Each runs the `_or_error` pipeline it names and collects the values, or stops at the first element that holds an error and returns that error instead. Nothing after the first error is examined. `ranges::to` does not do this: collecting a range of `expected<T, E>` into an `expected<vector<T>, E>` is what functional libraries call sequencing the range through the `expected` applicative, first error wins, and the standard has no spelling for it. `iconv_transcode_to_or_error` already answered the same question for the `iconv` adaptor; these are its WHATWG and codec-object counterparts. An encoding chosen at run time goes through the codec-object form with `whatwg::dynamic_decoder` or `whatwg::dynamic_encoder`.

0.5 Design Rationale

0.5.1 The Status Quo

`mbstowcs`, `iconv`, and `std::codecvt` are covered above, and the comparison tables show what each of them costs at the call site. The remaining option is to take a dependency: ICU, Boost.Text, and the like are capable, and heavyweight enough that not every project can afford one.

None of the four integrate with the ranges library. Converting between encodings requires extracting data from one container, passing it through a conversion API, and collecting into another container — losing the composability that makes ranges effective.

0.5.2 Why WHATWG?

The WHATWG Encoding Standard defines precise behavior for every legacy encoding encountered on the web. It gives exact byte-to-scalar mappings for all byte values, including every error case, and the Web Platform Tests [WPT-Encoding] pin that behavior across all major browsers.

The objection is real. WHATWG is a web specification, and it makes web-compatible choices that a general text library would not. It conflates distinct UTF-8 error conditions, it strips BOMs, and its label table exists to parse HTML `<meta>` tags. However, those choices are the ones that major browser engines already agree on and test, for the encodings that legacy data is actually written in. A general specification with no implementations to agree with would be worse.

Targeting it means a C++ program decodes a page the way the browser that fetched it did, and parses HTML and JSON with the same error handling.

0.5.3 Why Not `text_encoding::id`?

`std::text_encoding::id` is the obvious name for an encoding, and the library does not use it as its own. An `id` names an entry in the IANA registry, as "Context and Motivation" describes. It carries no decoding semantics. The registry says a document is Shift_JIS, and it does not say what `0x5C` decodes to.

The backend decides, and two backends given the same `id` can answer differently. `examples/text_encoding_two_answers.c` takes `id::ShiftJIS` and decodes the bytes `5C 7E` twice, once through WHATWG and once through iconv with the id's registered name. On Linux with GCC 16.2 and glibc 2.39 it prints:

```
WHATWG shift_jis: U+005C U+007E
iconv Shift_JIS  : U+00A5 U+203E
```

The Encoding Standard maps the single-byte range of `shift_jis` as ASCII. glibc's `SHIFT_JIS` maps it as JIS X 0201 Roman, so a backslash is a yen sign and a tilde is an overline. Neither is a bug. Each backend was asked what the name means, and answered for itself. The example is not a test, because the iconv answer depends on the platform's tables.

The label mapping differs too. WHATWG maps `latin1`, `iso-8859-1` and `ascii` to windows-1252, so `id::ISOLatin1` and `id::ASCII` resolve to `whatwg::encoding::windows_1252`. A program that passes `id::ISOLatin1` to WHATWG gets windows-1252, which is what a browser does with that label. The difference is deliberate. `text_encoding` matches aliases as Unicode Technical Standard #22 describes, and the Encoding Standard [WHATWG-Encoding] says of its own lookup:

> This is a more basic and restrictive algorithm of mapping labels to encodings than section 1.4 of Unicode Technical Standard #22 prescribes, as that is necessary to be compatible with deployed content.

So the library keeps its own enumeration. It is a closed set of specifications, and an `id` is only a name. The library bridges to `id` in both directions. `whatwg::get_encoding(text_encoding::id)` applies the WHATWG label lookup to the id's registered names, and `whatwg::to_text_encoding_id(e)` goes back through the encoding's WHATWG name. Three encodings have no id: `replacement`, `x_user_defined` and `x_mac_cyrillic`. So no cast between the two enumerations can be correct: several ids land on one encoding, and three encodings have no id to land on. For every other encoding the round trip returns the encoding it started with. Both functions are available only when `<text_encoding>` is, which is C++26.

0.5.4 Why Forward-Only Iterators?

The proposed decode, encode, and `iconv` transcode views all model `input_range`. That is the strongest category that can be implemented uniformly across all supported encodings, and the stateful multibyte codecs are what hold it there. The WHATWG decode and encode views preserve stronger properties when the codec and base range permit it: const-qualified iteration is supported for const-compatible bases, `common_range` and `borrowed_range` propagate from the base when implementable, and indexed codecs lift to `random_access_range`. The eager bulk helpers are layered on top of those pipelines when callers want collected results.

Legacy multibyte encodings like Shift_JIS, Big5, and GB18030 are fundamentally forward-only. Unlike UTF-8, where lead and continuation bytes occupy disjoint ranges, legacy encodings have overlapping byte values that can serve as either lead or trail bytes depending on context.

Consider Shift_JIS: the byte `0x81` is valid both as a lead byte starting a two-byte sequence and as a trail byte completing one. Given only a pointer into the middle of a byte stream, it is impossible to determine whether you are at a character boundary without scanning from the beginning.

Even with a known character boundary, stepping backward is impossible without external state. Take the byte sequence `0x81 0x81 0x81 0x40`. Read from a character boundary it is `0x81 0x81` and `0x81 0x40`: (U+FF1D FULLWIDTH EQUALS SIGN) then (U+3000 IDEOGRAPHIC SPACE). Read one byte late — the first `0x81` having been the trail byte of a character that began before it — what is left is `0x81 0x81` and a lone `0x40`: then '@'. Same four bytes, different text, and which reading applies depends on how many `0x81` bytes preceded them from the string's start.

WHATWG's state machines are designed as forward-only decoders, resetting their state at each character boundary, and bidirectional iteration would require caching all byte offsets during forward traversal. The encodings are like that. The implementation has no choice.

0.5.5 The Codec Protocol Is a Coalgebra

The codec concepts in Decoder and Encoder Concepts were arrived at one requirement at a time, and each is argued on its own below. They also have a shape, and naming it says why the pieces are the ones they are.

A decoder's `decode_one` takes a seed, the input position together with whatever state the codec carries, and produces one step: a scalar value and the advanced seed, an error and the advanced seed, or, with `is_empty`, the advanced seed alone. That is the step type of stream fusion [Stream-Fusion], `Done | Yield a s | Skip s`, and a function from a seed to one step is a coalgebra. `decode_view` is the unfold of that coalgebra, realised as an iterator rather than as a list: each increment runs the coalgebra until it yields, and the iterator's position is the seed. The companion paper on recursive tree algorithms [D4322] calls the operation `unfold_with`. There the embedding allocates one layer of a tree; here it advances an iterator.

Several decisions follow from the shape, and not only from the case made for each:

— *The state lives in the codec, and a codec is* ***semiregular****.* A coalgebra is a function of its seed, so the seed has to hold everything the next step depends on. That is why `begin()` starts from a fresh copy of the codec and the range carries nothing.
— ***is_empty*** *exists.* The coalgebra is one step, not a loop. An escape sequence that only changes the decoder's mode consumes input and yields nothing, and `Skip` is the honest result. Without it the codec would loop internally and the view would loop around it.
— ***flush*** *exists.* `Done` is the coalgebra's decision, not the sentinel's. A seed whose input is exhausted may still yield, as a decoder that stopped mid-sequence yields a truncation error, which is why the view asks `flush()` again after each engaged result until it declines.
— *Forward-only is structural.* An unfold produces a sequence one step at a time, and such a sequence has no random access. The `single_byte_` refinements are the one case where the coalgebra factors: when every byte decodes on its own, the view is a transform of the byte range, a map rather than an unfold, and random access survives. It is a property the codec declares, not one the view can supply.

An encoder's `encode_one` runs the other way, and the asymmetry between the two views has the same explanation. One scalar value becomes zero to eight bytes, so `encode_view` is a flattening, `concat_map` rather than `unfold`, and that is why it buffers where `decode_view` holds one element.

`transcode(from, to)` is specified as `decode(from) | encode(to)`, with no transcoder type. In these terms a transcoding pipeline run to completion is an unfold whose output is consumed by a fold, a hylomorphism, and the fusion law for that composition says the intermediate sequence of scalar values need never exist. Range laziness is what carries the law out, and the paper's claim that `whatwg::transcode<From, To>` materialises nothing is that law stated for this library. The `_or_error` views do not compose the same way, and that too is the shape: `unicode_scalar_range` asks for `char32_t`, so a decode error has to be settled before an encoder sees the value. Composing two error-producing stages is a monadic bind, not the functor-level composition `decode | encode` is, and the proposal declines to spell it. The bulk `_or_error` helpers in Bulk conversion to owned storage are the eager sequencing of one stage's results, which is as far as the composition goes.

0.6 Design

0.6.1 Overview

The proposal has three layers.

WHATWG. A closed set of encodings, each with a decoder, `whatwg::decoder<C>`, and, where the Encoding Standard defines one, an encoder, `whatwg::encoder<C>`. The Web Platform Tests pin their behavior, and all of it is `constexpr`. This is what almost every program wants.

Codec objects. The `text_decoder` and `text_encoder` concepts, and the `decode`, `encode` and `transcode(from, to)` adaptors over them. The WHATWG adaptors are specified as these adaptors applied to the library's own codec objects, so there is one decode view and one encode view, not two families. The layer is also the extension point for an encoding WHATWG does not define. We hope few programs need to write one.

System iconv. Everything else, with whatever semantics the platform has.

WHATWG plus iconv should cover the real world. The codec-object layer lets a program fill a gap without waiting for either.

The proposal adds five user-facing adaptor families:

— **whatwg::decode<C>**: Decodes a byte range to char32_t (Unicode scalar values). Decode errors are replaced with U+FFFD. It is the codec-object adaptor decode applied to whatwg::decoder<C>, so the result is a decode_view.

— **whatwg::encode<C>**: Encodes a char32_t range to bytes. Input is validated as UTF-32: an ill-formed code unit is replaced with U+FFFD, then encoded through C. A scalar value that C cannot represent is replaced with '?'. The WHATWG UTF-16BE and UTF-16LE encodings are decode-only in this interface, matching the Encoding Standard's lack of UTF-16BE/LE encoders.

— **whatwg::decode_or_error<C> / whatwg::encode_or_error<C>**: Error-reporting variants yielding expected<T, whatwg::error>. These are adaptor objects rather than separate view templates: each is the adaptor above with a different error_mode, so whatwg::decode_or_error<C> is decode_or_error applied to whatwg::decoder<C>, and produces a decode_view in report mode.

— **decode(d) / encode(e) / transcode(d, e)**: The same adaptors over any codec object, including a program's own. d models text_decoder and e models text_encoder; decode_or_error(d) and encode_or_error(e) are the report-mode forms. whatwg::decode<C> is decode(whatwg::decoder<C>{}), and whatwg::transcode<From, To> is transcode(whatwg::decoder<From>{}, whatwg::encoder<To>{}).

— **iconv_transcode_view / iconv_transcode(from, to, buf)** and **iconv_transcode_or_error_view / iconv_transcode_or_error(from, to, buf)**: Runtime-selected transcoding through the platform iconv implementation, using caller-provided staging storage.

Four eager bulk helpers, each specified as the pipeline it names:

— **whatwg::decode_to<C>(r) / whatwg::encode_to<C, Container>(r)**: Collect the decoded or encoded result into a container.

— **whatwg::decode_into<C>(r, out) / whatwg::encode_into<C>(r, out)**: Write the result through an output iterator.

Each has a form that takes the encoding as a value chosen at run time, whatwg::decode_to(r, e) and the rest, and decode_to and encode_to also take a label. These look at the encoding once and then run the same loop the compile-time form does. encode_to returns optional, empty when the encoding has no encoder.

Two codec objects for an encoding chosen at run time:

— **whatwg::dynamic_decoder{e} / whatwg::make_dynamic_encoder(e)**: Behave as whatwg::decoder<E> and whatwg::encoder<E> for the E equal to e. make_dynamic_encoder returns optional<dynamic_encoder>, empty when the encoding has no encoder, so the check a program needs is the call that creates the encoder. bytes | decode(whatwg::dynamic_decoder{e}) is a lazy char32_t range over an encoding named in the data.

And three utility entry points:

— **whatwg::transcode<From, To>**: Convenience composition of whatwg::decode<From> followed by whatwg::encode<To>, for To codecs that have a WHATWG encoder.

— **whatwg::transcode_string(source, from, to)**: Eager convenience helper for runtime-selected WHATWG transcoding by codec or by label. It returns an engaged optional<string> containing the converted bytes, including an empty string for an empty input, or nullopt if the target codec has no WHATWG encoder.

— **null_term_view / views::null_term**: Adapts a pointer to a null-terminated string into a range. views::null_term(cstr) | whatwg::decode<whatwg::encoding::utf_8> then composes.

0.6.2 The Encoding Enumeration

```
namespace whatwg {
enum class encoding {
  utf_8,

  // WHATWG special
  replacement,     // Always yields a single U+FFFD
```

```
  x_user_defined,  // Maps 0x80-0xFF to U+F780-U+F7FF

  // Single-byte legacy
  ibm866, iso_8859_2, iso_8859_3, iso_8859_4,
  iso_8859_5, iso_8859_6, iso_8859_7, iso_8859_8,
  iso_8859_8_i, iso_8859_10, iso_8859_13, iso_8859_14,
  iso_8859_15, iso_8859_16, koi8_r, koi8_u, macintosh,
  windows_874, windows_1250, windows_1251, windows_1252,
  windows_1253, windows_1254, windows_1255, windows_1256,
  windows_1257, windows_1258, x_mac_cyrillic,

  utf_16be,  // decode-only: WHATWG defines no UTF-16BE encoder
  utf_16le,  // decode-only: WHATWG defines no UTF-16LE encoder

  // CJK multibyte
  gbk,
  gb18030,
  big5,
  shift_jis,
  euc_jp,
  iso_2022_jp,
  euc_kr,
};
}
```

The synopsis in [transcode.whatwg.encoding] declares the enumerators without initializers, so their values are fixed by that declaration, as for any enumeration. Nothing in the proposal depends on them. The order is the Encoding Standard's table order, chosen for the reader, and the comments above are not part of the wording.

Each enumerator names the encoding of the same name in the WHATWG Encoding Standard, whose table of encodings is normative and is not restated here. An encoding is what that standard says it is, and a copy of the list in this paper would be a second one to keep current. `whatwg::encoding::replacement` is the standard's `replacement` encoding, which decodes any non-empty input to a single U+FFFD REPLACEMENT CHARACTER, and `whatwg::encoding::x_user_defined` is its `x-user-defined`.

The encoding is a template parameter and not a runtime value, so there is no per-element dispatch, and transcoding can run at compile time.

0.6.3 Label Lookup for Runtime Encoding Selection

The WHATWG Encoding Standard §4.2 defines over 200 string labels that map to canonical encodings. For example, `"shift_jis"`, `"sjis"`, `"x-sjis"`, `"ms_kanji"`, and `"csshiftjis"` all refer to the same Shift_JIS encoding. This mapping is essential for:

— Parsing `charset` attributes in HTML `<meta>` tags
— Parsing `charset` parameters in `Content-Type` headers
— Web-compatible encoding sniffing algorithms

A `whatwg::get_encoding` function provides runtime codec selection:

```
constexpr std::optional<whatwg::encoding> whatwg::get_encoding(std::string_view label) noexcept;
```

The function performs ASCII case-insensitive matching and strips leading/trailing ASCII whitespace, per WHATWG §4.2:

```
static_assert(whatwg::get_encoding("UTF-8") == whatwg::encoding::utf_8);
static_assert(whatwg::get_encoding("utf-8") == whatwg::encoding::utf_8);
static_assert(whatwg::get_encoding("  utf-8  ") == whatwg::encoding::utf_8);
static_assert(whatwg::get_encoding("shift_jis") == whatwg::encoding::shift_jis);
static_assert(whatwg::get_encoding("SJIS") == whatwg::encoding::shift_jis);
```

```cpp
static_assert(whatwg::get_encoding("x-sjis") == whatwg::encoding::shift_jis);
static_assert(whatwg::get_encoding("unknown") == std::nullopt);
```

When the encoding arrives as metadata, the program does not dispatch on it. A MIME part names its charset in its headers, and its reader wants the text as code points:

```cpp
std::string_view charset = part.charset();
if (auto text = whatwg::decode_to(part.body(), charset)) {
    consume(*text);  // vector<char32_t>
}
```

`decode_to` returns `nullopt` when the label names no encoding. A body that arrives as a range, out of a base64 decoder say, need not be collected first:

```cpp
if (auto e = whatwg::get_encoding(charset)) {
    for (char32_t c : body | base64_decode | decode(whatwg::dynamic_decoder{*e}))
        consume(c);
}
```

And `whatwg::transcode_string(body, charset, "utf-8")` goes straight to bytes in another encoding. The reference implementation has a small MIME reader among its examples that does all of this, with test messages in ISO-2022-JP, Shift_JIS, windows-1252, UTF-8 and KOI8-R.

0.6.4 Error Handling

Two WHATWG error handling strategies are provided:

1. **Replacement mode** (`whatwg::decode`, `whatwg::encode`, and `whatwg::transcode`): Invalid byte sequences decode to U+FFFD. Ill-formed UTF-32 input is replaced with U+FFFD before encoding, while a scalar value the target encoding cannot represent is replaced with `'?'`. This follows the WHATWG replacement model and is appropriate for most text processing where halting on errors is undesirable.
2. **Report mode** (`whatwg::decode_or_error`, `whatwg::encode_or_error`): Each output element is `expected<T, whatwg::error>`. This allows callers to log errors, apply custom replacement, or abort processing.

For the WHATWG encodings, the error enumeration provides specific failure reasons:

```cpp
namespace whatwg {
enum class error {
  invalid_byte,         // Byte not valid in this position
  truncated_sequence,   // End of input mid-character
  overlong_encoding,    // UTF-8 overlong sequence
  surrogate_code_point,// Decoded or UTF-32 input value is a surrogate
  out_of_range,         // Decoded or UTF-32 input value is above U+10FFFF
  unmapped_codepoint,   // Encode: no mapping exists
};
}
```

0.6.4.1 Relationship to `utf_transcoding_error` (P2728)

P2728 (beman.utf_view) defines `utf_transcoding_error` for its `_or_error` views. The two enumerations are distinct types. Most values correspond directly:

`whatwg::error`	`utf_transcoding_error` (P2728)	Notes
`truncated_sequence`	`truncated_utf8_sequence`	Same concept; `whatwg::error` applies to all codecs, not just UTF-8
`overlong_encoding`	`overlong`	Identical semantics

whatwg::error	utf_transcoding_error (P2728)	Notes
surrogate_code_point	encoded_surrogate	Identical semantics for UTF-8 and UTF-32
out_of_range	out_of_range	Identical semantics
invalid_byte	invalid_utf8_leading_byte + unexpected_utf8_continuation_byte	P2728 distinguishes two invalid-byte cases; WHATWG conflates them because its spec produces U+FFFD for both without differentiating
unmapped_codepoint	*(no equivalent)*	See below
(not needed)	unpaired_high_surrogate / unpaired_low_surrogate	UTF-16 code-unit-level concepts; this library operates on byte streams with explicit endianness (whatwg::encoding::utf_16be/utf_16le), so there is no paired/unpaired distinction at the byte level

unmapped_codepoint is the reason reusing utf_transcoding_error directly is not possible. P2728's Unicode-only encoders can encode every Unicode scalar value, so P2728 never needs to report "no mapping exists for this code point." Legacy codecs cannot: windows-1252 has no representation for U+4E2D ( ), Shift-JIS has no representation for U+00E9 (é), and so on for most of the Unicode range. whatwg::encode_or_error<whatwg::encoding::windows_1252> must be able to report this condition through the same error type as whatwg::encode_or_error<whatwg::encoding::utf_8>, so the error type must accommodate it uniformly across the WHATWG encode surface.

The iconv adaptors use a separate iconv_error enumeration:

```
enum class iconv_error {
  invalid_sequence,
  incomplete_sequence,
  output_full,
  open_failed,
  system_error,
};
```

Two error vocabularies in one header wants a defence, so here it is. They are separate because the failures are not the same failures. whatwg::error names what the Encoding Standard says went wrong in a byte sequence this library decoded, and every one of its enumerators corresponds to a step in an algorithm the paper specifies. iconv_error names what the platform reported about a conversion this library did not perform: EILSEQ, EINVAL, E2BIG, a descriptor that would not open, or an unexpected system failure. Those codes say *that* a byte sequence is not valid, not why, so the WHATWG error vocabulary would be claiming knowledge the operating system does not return. A single enumeration would have to either drop that distinction — reporting invalid_byte for a failure whose meaning is "the platform's tables say so", with no algorithm behind it a reader can consult — or carry both sets, which is two vocabularies with one name. Nothing composes them, either: the iconv adaptor is byte byte and does not appear in a decode | encode pipeline, so no expression ever has to reconcile the two.

This is separate from the unification of the _or_error views. Those pairs collapsed into one view template parameterized on error_mode, which selects whether errors are *reported* or replaced. What an error *is* remains the codec family's own question.

0.6.5 Concepts

Two concepts constrain the input ranges:

```
template <typename R>
concept legacy_byte_range =
    ranges::range<R> &&
    !is_array_v<remove_cvref_t<R>> &&
```

```
    legacy-byte-type<remove_cv_t<ranges::range_value_t<R>>>;

template <typename R>
concept unicode_scalar_range =
    ranges::input_range<R> &&
    !is_array_v<remove_cvref_t<R>> &&
    same_as<remove_cv_t<ranges::range_value_t<R>>, char32_t>;
```

where *legacy-byte-type* is an exposition-only concept satisfied by `char`, `signed char`, `unsigned char` and `byte`. [transcode.reqs] is the normative text; this block restates it.

`unicode_scalar_range` is intentionally a type-level concept. It identifies the library's scalar-value interchange representation, but it cannot prove that each `char32_t` object is a Unicode scalar value. Encode adaptors therefore interpret their input as UTF-32 and perform the conversion to scalar values above codec dispatch, just as the Web Platform converts strings before invoking a WHATWG encoder. Surrogates and values above U+10FFFF become U+FFFD in replacement mode or their corresponding errors in report mode.

The two are not symmetric: `legacy_byte_range` requires `ranges::range`, where `unicode_scalar_range` requires `ranges::input_range`. Nothing in the proposal reaches the weaker requirement — every view constrained on `legacy_byte_range` constrains its base on `input_range` as well, and the bulk helpers forward to those views — so aligning the two on `input_range` would change no program. It is called out because the two declarations side by side invite the question.

Raw arrays are explicitly rejected to prevent silent inclusion of null terminators. Use `views::null_term` for null-terminated strings or wrap counted byte buffers in `span`.

The `char8_t` type is not accepted because it implies UTF-8 encoding, conflicting with the codec parameter. Similarly, `wchar_t` is rejected due to its platform-dependent size.

0.6.6 Decoder and Encoder Concepts

The range concepts constrain what goes into a view. These constrain the codec object the view is built over:

```
struct decode_result {
    char32_t      code_point{};
    whatwg::error error{};
    bool          is_error{false};
    bool          is_empty{false};
};

struct encode_result {
    unsigned char bytes[8]{};
    int           count{0};
    bool          is_error{false};
};

template <typename C>
concept text_decoder = semiregular<C> &&
    requires(C& c, const unsigned char*& iter, const unsigned char* sentinel) {
        { c.decode_one(iter, sentinel) } -> same_as<decode_result>;
    };

template <typename C>
concept text_encoder = semiregular<C> && requires(C& c, char32_t cp) {
    { c.encode_one(cp) } -> same_as<encode_result>;
};

template <typename C>
concept flushable_text_decoder = text_decoder<C> && requires(C& c) {
    { c.flush() } -> same_as<optional<decode_result>>;
```

```
};

template <typename C>
concept bom_aware_text_decoder =
    text_decoder<C> && requires(C& c, const unsigned char*& iter, const unsigned char* sentinel) {
        { c.decode_first(iter, sentinel) } -> same_as<decode_result>;
    };

template <typename C>
concept flushable_text_encoder = text_encoder<C> && requires(C& c) {
    { c.flush() } -> same_as<optional<encode_result>>;
};

template <typename C>
concept single_byte_text_decoder = text_decoder<C> && requires(const C& c, unsigned char byte) {
    { c.decode_byte(byte) } -> same_as<char32_t>;
};

template <typename C>
concept single_byte_text_encoder = text_encoder<C> && requires(const C& c, char32_t scalar) {
    { c.encode_byte(scalar) } -> same_as<optional<unsigned char>>;
};
```

[transcode.codec.reqs] is the normative text.

A codec is a value. It is `semiregular` because a view owns one and copies it, and it holds its own state, so every `begin()` starts from a fresh copy. The state lives in the codec and not in the range.

`decode_one` reads from an iterator up to a sentinel, advances the iterator past what it consumed, and returns one `decode_result`. `is_empty` means the call consumed input and produced nothing, as an escape sequence that only changes the decoder's mode does. The view asks again. A decoder that also models `bom_aware_text_decoder` has `decode_first` called for the first element in place of `decode_one`, which is how the UTF decoders drop a leading U+FEFF.

An encoder never sees a bad input. The view validates its UTF-32 before it calls `encode_one`, so the argument is always a Unicode scalar value. What the encoder can fail on is a scalar value its encoding has no representation for. In replacement mode the view then yields the bytes the encoder supplied in `bytes` and `count`, and `'?'` if it supplied none.

`flush` is how a stateful codec settles at the end of input. The view calls it when the input is exhausted, and again after each engaged result, until it returns `nullopt`. ISO-2022-JP is the motivating case: an encoder that has shifted out of ASCII owes the escape sequence that shifts back. A decoder owes a truncation error for a sequence that ended mid-way.

The single-byte refinements exist so that random access survives the generic path. When every byte decodes independently, the nth element is the nth byte, and a view over a `single_byte_text_decoder` is a `random_access_range`. The same holds for a `single_byte_text_encoder` and the scalar values going in. `single_byte_text_encoder`, with random-access encode, is the part of this section that SG16 may prefer to drop. The decode side is what the WHATWG single-byte encodings need, and the encode side is there to keep `whatwg::encode<C>` a `random_access_range` for them.

These concepts are not in `whatwg::`, because they describe the generic protocol, and any codec can model them. The one-to-one mapping to the Encoding Standard is on the codec objects. As the Standard puts it, "each encoding has an associated decoder and most of them have an associated encoder". So `whatwg::decoder<C>` models `text_decoder` for every `whatwg::encoding`, and `whatwg::encoder<C>` exists only for the encodings that have an encoder. The error vocabulary is still `whatwg::error`, because the protocol adopts WHATWG's error model.

The names split into verb and noun. `whatwg::decode<C>` is the adaptor, and it is `decode(whatwg::decoder<C>{})`. `whatwg::transcode<From, To>` is `transcode(whatwg::decoder<From>{}, whatwg::encoder<To>{})`. There is no transcoder type.

0.6.6.1 A Codec the Library Does Not Ship

WHATWG has no ISO-8859-1. Its `latin1` and `iso-8859-1` labels mean windows-1252, which differs in `0x80` to `0x9F`. A program that needs the ISO encoding has a real gap, and fills it by modeling the two concepts:

```
struct iso_8859_1 {
    template <input_iterator I, sentinel_for<I> S>
    constexpr decode_result decode_one(I& current, S end) const;

    constexpr encode_result encode_one(char32_t cp) const;
};

template <input_iterator I, sentinel_for<I> S>
constexpr decode_result iso_8859_1::decode_one(I& current, S end) const {
    const auto byte = static_cast<unsigned char>(*current);
    ++current;
    return {char32_t{byte}, {}, false};
}

constexpr encode_result iso_8859_1::encode_one(char32_t cp) const {
    if (cp > 0xFF)
        return {{}, 0, true};
    encode_result result{};
    result.bytes[0] = static_cast<unsigned char>(cp);
    result.count    = 1;
    return result;
}

static_assert(text_decoder<iso_8859_1> && text_encoder<iso_8859_1>);
```

`char32_t` is the seam between codecs, so the custom decoder pipes into a WHATWG encoder and a WHATWG decoder into the custom encoder:

```
// ISO-8859-1 to UTF-8
for (char b : latin1 | decode(iso_8859_1{}) | whatwg::encode<whatwg::encoding::utf_8>)
    out.push_back(b);

// UTF-8 to ISO-8859-1; a scalar value with no ISO-8859-1 byte becomes '?'
for (char b : utf8 | whatwg::decode<whatwg::encoding::utf_8> | encode(iso_8859_1{}))
    out.push_back(b);

// windows-1252 to ISO-8859-1 in one pipeline
for (char b : bytes | transcode(whatwg::decoder<whatwg::encoding::windows_1252>{}, iso_8859_1{}))
    out.push_back(b);
```

The example is `examples/paper_custom_codec.cpp`. It is compiled and run as a test, and its conversions are also `static_asserted`, so the claims above are checked at compile time.

0.6.7 Constexpr Support

All views are `constexpr`-enabled. Decoding and encoding tables are compile-time constants. Decoding therefore runs in a constant expression:

```
constexpr auto decoded = []() consteval {
    constexpr char bytes[] = {'\xC3', '\xA9'};
    std::span<const char> sp(bytes, 2);
    return *(sp | whatwg::decode<whatwg::encoding::utf_8>).begin();
}();
static_assert(decoded == U'é');
```

0.6.8 Pipe Syntax

The views support both function call and pipe syntax:

```
// Function call
auto v1 = whatwg::decode<whatwg::encoding::utf_8>(bytes);

// Pipe
auto v2 = bytes | whatwg::decode<whatwg::encoding::utf_8>;

// Composition
auto v3 = bytes
  | whatwg::decode<whatwg::encoding::shift_jis>
  | whatwg::encode<whatwg::encoding::utf_8>;

// Convenience closure
auto v4 = bytes | whatwg::transcode<whatwg::encoding::shift_jis, whatwg::encoding::utf_8>;
```

0.6.9 Range Properties and Const Support

All of the views are valid `input_ranges`, and the WHATWG views preserve stronger range properties where they can.

— **Const iteration**: `whatwg::decode`, `whatwg::encode`, and their `_or_error` variants support `begin() const` and `end() const` when the underlying range supports compatible const iteration. This includes owning views after construction.

— **`common_range` and `borrowed_range`**: These properties are preserved from the base range when the codec can support them.

— **Indexed codecs**: Single-byte and other indexed codecs can provide `random_access_range` behavior for both regular and `_or_error` WHATWG views. Stateful codecs such as UTF-8 and ISO-2022-JP remain forward-only or input-only where required by their decoding model.

0.6.10 User-Facing Examples

The separate decode and encode entry points make the pipeline direction explicit:

```
std::vector<char> utf8 = {'H', 'i', '\xE2', '\x82', '\xAC'};

auto code_points = utf8 | whatwg::decode<whatwg::encoding::utf_8>;
auto round_trip  = code_points | whatwg::encode<whatwg::encoding::utf_8>;
```

Collecting into owned storage uses `std::ranges::to`, or the named helpers:

```
std::string bytes = "caf\xE9";
auto scalars = bytes
  | whatwg::decode<whatwg::encoding::windows_1252>
  | std::ranges::to<std::vector<char32_t>>();

auto utf8 = scalars
  | whatwg::encode<whatwg::encoding::utf_8>
  | std::ranges::to<std::string>();

auto scalars2 = whatwg::decode_to<whatwg::encoding::windows_1252>(bytes);
auto utf8_2   = whatwg::encode_to<whatwg::encoding::utf_8, std::string>(scalars2);
```

Writing into an output iterator uses `std::ranges::copy`, or `decode_into` and `encode_into`:

```
std::vector<char32_t> scalars;
std::ranges::copy(bytes | whatwg::decode<whatwg::encoding::utf_8>,
                  std::back_inserter(scalars));
```

```
std::string encoded;
std::ranges::copy(scalars | whatwg::encode<whatwg::encoding::utf_8>,
                  std::back_inserter(encoded));

whatwg::decode_into<whatwg::encoding::utf_8>(bytes, std::back_inserter(scalars));
whatwg::encode_into<whatwg::encoding::utf_8>(scalars, std::back_inserter(encoded));
```

None of this needs the library to allocate. The WHATWG views allocate nothing; the containers in these examples are the caller's choice, not the library's. A program that owns a fixed buffer writes into it, and bounds the write with `views::take`:

```
std::size_t decode_into_buffer(
    std::string_view bytes,
    std::array<char32_t, 256>& buf) {
  auto [in, out] = std::ranges::copy(
      bytes
        | whatwg::decode<whatwg::encoding::utf_8>
        | std::views::take(buf.size()),
      buf.begin());
  return static_cast<std::size_t>(out - buf.begin());
}
```

The convenience `whatwg::transcode<From, To>` adaptor is useful when the intermediate Unicode range is not otherwise needed:

```
std::string latin1 = "caf\xE9";
auto utf8 = latin1
  | whatwg::transcode<whatwg::encoding::windows_1252, whatwg::encoding::utf_8>
  | std::ranges::to<std::string>();
```

Null-terminated inputs compose through `views::null_term`:

```
const char* cstr = "hello";
std::u32string scalars = views::null_term(cstr)
  | whatwg::decode<whatwg::encoding::utf_8>
  | std::ranges::to<std::u32string>();
```

Runtime labels can be resolved through WHATWG's alias table before dispatch:

```
if (auto utf8 = whatwg::transcode_string(bytes, "shift_jis", "utf-8")) {
  consume(*utf8);
}
```

For encodings outside the WHATWG set, the `iconv` adaptor provides a runtime-selected escape hatch:

```
std::array<char, 256> buffer{};

std::vector<char> utf32le = input
  | iconv_transcode("UTF-8", "UTF-32LE", std::span(buffer))
  | std::ranges::to<std::vector>();
```

0.7 Design Decisions

0.7.1 Byte-Like Types, Not Character Types

The `legacy_byte_range` concept accepts `char`, `signed char`, `unsigned char`, and `std::byte` — the types that represent raw octets in C++. It explicitly rejects:

— `char8_t`: Implies UTF-8 encoding, which conflicts with explicit codec selection. If you have `char8_t` data, you already know it's UTF-8 — passing it to `whatwg::decode<whatwg::encoding::shift_jis>` would be a logic error.

— `wchar_t`: Platform-dependent width (16-bit on Windows, 32-bit on POSIX) makes it unsuitable as an interchange format. It also implies "already decoded" where what is wanted is "raw bytes."

— `char16_t` / `char32_t`: These are Unicode code unit types. A range of `char16_t` is already decoded data that might need re-encoding.

This design reflects where encoded text comes from: network sockets (`recv` returns bytes), file I/O (`read` returns bytes), legacy APIs (return `char*`), and modern buffer types (`std::byte`). The views take the bytes as they arrive, with no conversion step first.

Rejecting `char8_t` inconveniences anyone holding a `u8string` that is not in fact UTF-8, which does happen when a codebase uses `u8string` as its default string type. However, accepting it would mean accepting a type whose purpose is to state an encoding, and then ignoring what it states. Such a caller can use `as_bytes` and make that reinterpretation visible.

```
// All of these work:
std::vector<char> v1;
std::vector<unsigned char> v2;
std::vector<std::byte> v3;
std::span<const signed char> s;

v1 | whatwg::decode<whatwg::encoding::utf_8>;  // OK
v2 | whatwg::decode<whatwg::encoding::utf_8>;  // OK
v3 | whatwg::decode<whatwg::encoding::utf_8>;  // OK
s  | whatwg::decode<whatwg::encoding::utf_8>;  // OK

// These are rejected at compile time:
std::u8string u8s;
u8s | whatwg::decode<whatwg::encoding::utf_8>;  // Error: char8_t implies UTF-8
```

0.7.2 Template Parameter vs Runtime Selection

Making the codec a template parameter gives inlined codec logic, no dispatch per element, and `constexpr` transcoding.

What it gives up is the common case: the encoding usually arrives as a string from an HTTP header or a `<meta>` tag, and a template parameter cannot be spelled from a runtime string. However, the dispatch has to happen exactly once, at the point where the label is resolved, and it is a `switch` over an enumeration. The library writes that `switch` so that programs do not have to, in two forms. The eager forms, `whatwg::transcode_string` and the runtime overloads of `decode_to`, `encode_to`, `decode_into` and `encode_into`, take the encoding as a value or a label, look at it once, and then run the loop the compile-time form would. The codec objects `whatwg::dynamic_decoder` and `whatwg::dynamic_encoder` give a lazy view over an encoding chosen at run time; they look at the encoding on every call, which is the cost of not knowing it, and a caller pays it only by choosing them. Neither form gives up anything for the caller who knows the encoding at compile time.

We measured what that costs. The reference implementation has a dispatch-overhead benchmark (pull request 128 on the implementation repository) that counts instructions instead of timing them. Dispatching by encoding at run time costs about 121 instructions per call, whatever the encoding and whatever the size of the input. That is an upper bound, because it includes the wrapper in `whatwg::transcode_string`. Each `whatwg::get_encoding` label lookup costs about 550 instructions. Together they are under 1% of the work on whole corpora, and 3-5% on 32-byte inputs.

So a runtime caller pays almost nothing for the template parameter. Dispatching on every code point instead would give up `constexpr` decoding for all callers to spare one `switch` for some of them, and the `switch` is the part that costs almost nothing.

0.7.3 The `whatwg` Namespace

The WHATWG surface is in `std::whatwg`, so that its names are the Encoding Standard's own: `whatwg::encoding`, `whatwg::decoder<C>`, `whatwg::encoder<C>`, `whatwg::get_encoding`, `whatwg::bom_sniff`.

The generic plumbing stays in `std`. That includes the codec concepts, `text_decoder`, `text_encoder` and their refinements, because they describe the pluggable protocol and not WHATWG.

This is a choice about how many names the proposal adds to `std`, and it is open. The paper asks for direction, below.

It has a cost. The enum-keyed adaptors, `whatwg::decode<C>`, and the codec-object adaptors, `decode(d)`, have the same name one namespace apart. A variable template and a function template cannot share a scope, so they cannot live in one. A program that applies `using namespace` to both gets an ambiguity on the unqualified name. Distinct spellings, such as `decode_with(d)` for the codec-object adaptor, would remove the collision and cost a name in `std`.

`single_byte_text_encoder` is in the same position. It exists only to keep random access on the encode path, and it could be dropped along with it.

0.7.4 char32_t as the Interchange Type

Unicode scalar values are represented as single `char32_t` values. This simplifies composition, since a `char32_t` stream can be fed to any encoder without reparsing, and makes individual code point inspection trivial.

The type is an interchange representation. Nothing about `char32_t` validates what it holds. Decode views produce scalar values before an encoder sees them; a range of `char32_t` supplied from outside a decode pipeline is validated as UTF-32 by the encode view. Ill-formed code units are replaced with U+FFFD or reported, according to the view's error mode, before the selected codec is invoked.

UTF-32 is the wasteful choice. A `char32_t` per scalar is four bytes where UTF-8 would often use one. However, these are lazy views, so nothing is stored unless a caller collects it, and a single code point is trivial to inspect. All other interchange types are worse.

0.7.5 Separation of Decode and Encode

Rather than a single primary "transcode from A to B" adaptor, decode and encode are separate operations composed via pipes. This enables:

— Processing Unicode without re-encoding: `bytes | whatwg::decode<whatwg::encoding::utf_8> | filter(...)`
— Encoding to multiple targets: `codepoints | tee(whatwg::encode<whatwg::encoding::utf_8>(...), whatwg::en`
— Clear mental model: decode produces `char32_t`, encode consumes `char32_t`

The `whatwg::transcode<From, To>` convenience closure is layered on top of this composition model for the common case where the intermediate `char32_t` stream is not observed. The eager bulk helpers are layered on the same decode and encode semantics, but package the common collection patterns directly.

0.7.6 Leading BOM Stripping for UTF Decoders

The WHATWG UTF decoders strip a leading BOM when it matches the selected codec. This matches browser behavior and the Web Platform Tests [WPT-Encoding] for UTF-8, UTF-16LE, and UTF-16BE.

When the codec is not yet known, a separate `whatwg::bom_sniff` helper can examine the byte stream for a BOM before dispatch:

```
if (auto enc = whatwg::bom_sniff(bytes)) {
  // Dispatch according to the detected BOM.
}
```

Non-matching BOMs are not stripped. For example, a UTF-8 decoder does not remove a UTF-16LE BOM prefix.

`whatwg::bom_sniff` is the byte order mark half of the Encoding Standard's "decide the fallback encoding" step. What to do when there is no mark is a question about the document and its transport, which a library cannot answer. The mark itself is not consumed, because a decode view strips a leading U+FEFF for the UTF codecs on its own, so the range can be passed on unchanged.

`R` is required to be a `forward_range` for that last sentence to be true. Examining the mark means advancing an iterator over it, and on a single-pass range those bytes are not there afterwards. Sniffing a stream and

then decoding it would drop the prefix, and sniffing a stream with no mark, the common case, would drop the first two bytes of the document. Nothing in the signature would say so. A caller holding an `input_range` has to materialize enough of it to look twice.

0.7.7 No Normalization

Unicode normalization (NFC, NFD, NFKC, NFKD) is orthogonal to encoding. A `normalize_view<form>` is an obvious future addition, but not part of this proposal.

0.8 Methods

0.8.1 Authoritative Source Acquisition

The reference implementation does not hand-transcribe encoding tables or conformance fixtures. Instead, it stores pristine upstream source material for both the WHATWG Encoding Standard indexes [WHATWG-Encoding] and the relevant Web Platform Tests (WPT) [WPT-Encoding] under version control, together with provenance metadata and checksums.

Keeping the upstream bytes ties the implementation to the same normative and de facto-interoperability sources that define browser behavior. It also makes source updates auditable during review, because regenerated artifacts can be compared against the exact upstream inputs that produced them. And it avoids network access during normal builds and test runs; regeneration is an explicit maintenance step rather than a hidden part of compilation.

0.8.2 Python Preprocessing for Tables and Conformance Vectors

The project uses small Python helpers to transform upstream WHATWG and WPT data into C++-friendly artifacts. The generators are checked in.

For codec tables, Python scripts download the WHATWG index files, record provenance, and generate checked-in lookup tables for single-byte and multibyte codecs. This avoids manually maintaining large arrays of code points while keeping the generated results reviewable and deterministic.

For conformance tests, a separate Python generator parses the JavaScript-authored WPT fixtures and emits C++ headers containing byte sequences, expected Unicode scalar values, and descriptive labels. That translation step is necessary because the authoritative tests are written for the web platform rather than for C++. By converting them once into ordinary C++ data, the test suite can run as native unit tests without embedding a JavaScript engine or reimplementing the WPT harness at runtime.

The generators are themselves unit-tested. That keeps the extraction logic honest: if WPT uses surrogate pairs, replacement characters, BOM-sensitive cases, fatal-mode expectations, or codec-specific fixture shapes, those parsing rules are validated independently from the transcoder under test. For the `USVString` surrogate vectors, the Python parser combines surrogate pairs into one `char32_t` and leaves lone surrogates for the C++ view's UTF-32 validation. Consequently the astral vector is not a surrogate-validation case in C++, and a `char32_t` range cannot model a pair split across chunks.

0.9 Testing

0.9.1 Web Platform Tests

The implementation is validated against the WHATWG Web Platform Tests (WPT) [WPT-Encoding] for text encoding. These tests provide:

— Decode tests for each codec with representative byte sequences
— Error handling tests verifying U+FFFD replacement behavior
— Edge cases for truncated sequences, overlong UTF-8, and UTF-32 validation, including vectors derived from WPT's `USVString` coercion test rather than from a WHATWG encoder algorithm
— BOM handling: the UTF decoders strip a leading BOM that matches the selected codec, and the WPT vectors for that are what pin it

Test vectors are extracted from WPT and converted to C++ data structures using the preprocessing pipeline described in Methods. The implementation is checked against browser behavior automatically, with the executable tests entirely within the native C++ test harness.

0.9.2 Negative Compile Tests

Every concept constraint has a corresponding negative compile test — a `.cpp` file that must fail to compile with a specific diagnostic. This ensures:

— Raw arrays are rejected with a clear error message
— `char8_t` ranges are rejected (wrong for explicit codec selection)
— `wchar_t` ranges are rejected (non-portable width)
— Non-input ranges are rejected

0.9.3 Consteval Tests

Every operation that should be `constexpr` has a `consteval` test verifying compile-time evaluation:

```
constexpr auto result = []() consteval {
    constexpr char bytes[] = {'\xE2', '\x82', '\xAC'};
    std::span<const char> sp(bytes, 3);
    return *(sp | whatwg::decode<whatwg::encoding::utf_8>).begin();
}();
static_assert(result == U'€');
```

0.9.4 Roundtrip Tests

For codecs where encode is defined, roundtrip tests verify that `encode(decode(bytes)) == bytes` for all valid inputs, and that `decode(encode(codepoints)) == codepoints` for all mapped code points.

Bulk collection through `ranges::to` and output-iterator paths via `ranges::copy` are covered alongside the named helpers that forward to them.

0.9.5 Order-Sensitive Inputs

A roundtrip is not a sufficient test of anything that carries state or order. A codec with no state roundtrips even if the view drops codec state, a decoder that ignores `is_empty` or calls `flush` once too few roundtrips every input with no escape sequence and no truncated tail, and `decode(encode(x)) == x` holds for a helper that reports the last error instead of the first. So every test of a stateful codec, of `flush` and `is_empty` handling, or of an `_or_error` helper uses an input for which a plausible wrong implementation gives a visibly different answer: the Shift_JIS `81 81 81 40` sequence above, whose reading depends on where the boundary falls; an ISO-2022-JP text that ends mid-escape, so a missing `flush` is a missing error; two errors of different kinds in one input, so first-versus-last is caught; and a codec that counts its calls through a pointer the test owns, so "stopped at the first error" is observed rather than assumed. The rule is borrowed from the tree algorithms paper's test discipline [D4322], where a monotone, idempotent test function could not distinguish a correct fold from a plausible wrong one and a non-idempotent one could.

0.10 Impact on the Standard

This proposal adds new headers and does not modify existing standard library components. It is a pure extension.

0.10.1 Feature Test Macro

```
#define __cpp_lib_transcode_view 202XXXL
```

0.10.2 Headers

```
#include <transcode>   // whatwg::decode, whatwg::encode, whatwg::transcode, transcode,
                       // iconv_transcode, whatwg::transcode_string
#include <null_term>   // null_term_view
```

Or combined into `<ranges>` additions.

0.11 Open Questions for SG16

There are three.

The U+FFFD reservation. The random-access custom codec protocol, `single_byte_text_decoder`, reserves U+FFFD as the in-band signal that a byte `0x80` or above decodes to nothing. That keeps `decode_byte` cheap, but it prevents a custom codec from mapping such a byte to the actual U+FFFD scalar: replacement mode returns U+FFFD, while error-reporting mode calls the same value `invalid_byte`. Should the protocol retain that reservation? The alternatives are to return a result type from `decode_byte`, add a separate validity query, or keep and document the present restriction. This paper does not choose between them before SG16 review: no WHATWG single-byte codec needs the foreclosed mapping, and changing the indexed path for a hypothetical extension is a change the group should weigh.

The `whatwg` namespace. The WHATWG names are in `std::whatwg` (The `whatwg` Namespace). Should the proposal add that many names to `std`, and under that namespace? The alternatives are to keep it, to flatten the names into `std` with a `whatwg_` prefix, or to keep the namespace and give the codec-object adaptors distinct names such as `decode_with(d)`, so that a use-directive on both namespaces is not ambiguous.

`single_byte_text_encoder`. It exists only to keep `whatwg::encode<C>` a `random_access_range` for the single-byte encodings. Should it stay, or should random-access encode be dropped, and the concept with it? Dropping it removes a public name and `random_access_encode_view`, and `whatwg::encode<C>` becomes a forward range for every encoding.

0.12 Summary of the Design

The decisions embodied in the design, and in the wording that follows:

- **There are three layers**: the WHATWG encodings, codec objects, and system `iconv`. The WHATWG adaptors are the codec-object adaptors applied to `whatwg::decoder<C>` and `whatwg::encoder<C>`, and a program can supply a codec the library does not ship.
- **Decode and encode are separate adaptors**, composed by pipe. `transcode(from, to)` and `whatwg::transcode<From, To>` are the composed convenience for when the intermediate `char32_t` stream is not observed.
- **The codec is a template parameter for the WHATWG adaptors, and a value for codec objects.** Codec logic inlines, nothing dispatches per element, and transcoding runs at compile time. Runtime selection happens once, where the label is resolved, through `whatwg::get_encoding` and `whatwg::transcode_string`, and it costs about 121 instructions a call and about 550 a lookup.
- **The codec concepts are generic.** `text_decoder` and `text_encoder`, with the `flushable_text_decoder`, `bom_aware_text_decoder`, `flushable_text_encoder` and `single_byte_` refinements, are in `std`; the WHATWG names are in `std::whatwg`, which is an open question.
- **`char32_t` is the interchange type.** Nothing about it validates what it holds, so a `char32_t` range from outside a decode pipeline is validated as UTF-32 by the encode view.
- **Input is byte-like or it is rejected**: `char`, `signed char`, `unsigned char`, `std::byte`. `char8_t`, `wchar_t` and raw arrays do not participate.
- **Errors come in two vocabularies**, `whatwg::error` and `iconv_error`, because the failures are not the same failures. Whether an error is reported or replaced is one view template parameterized on `error_mode`.
- **The views model `input_range`** and preserve more where the codec and base permit, up to `random_access_range` for indexed codecs. That comes from the codec-object views, `decode_view` and `encode_view`, which are also `borrowed_range` when their base is.
- **Four named bulk helpers are proposed**, specified as the pipelines they name. They add no semantics beyond those pipelines; they make the common operation discoverable.

The `iconv` adaptor is proposed here, with the portable WHATWG facilities, rather than on its own. Its conversion-descriptor handle cannot be named entirely in ISO C++ terms without either introducing an opaque handle abstraction or moving the adaptor to a POSIX-conditional document. It is useful implementation experience and a valuable comparison point, and the shared interface is best considered as a whole, so it belongs here through SG16 design review. Its wording is [transcode.iconv], written last and resting on nothing the other clauses need, so splitting it later renumbers nothing.

The objection to specifying it is that POSIX `iconv` is implementation-defined across glibc, musl and the BSDs, so a view over it specifies whatever the platform does. That is true, and the wording does not pretend

otherwise: what it specifies is what the *adaptor* guarantees — one conversion descriptor per `begin`, closed by the iterator that owns it, and input the conversion refuses either skipped or surfaced as an `iconv_error` — and it leaves the conversion to the implementation, as `<locale>` leaves a locale's tables to it. That guarantee is the reason to propose it at all: the C interface hands a program a descriptor it must remember to close, and an adaptor is where that stops being the program's problem. It also does not restate a table it does not own.

The iconv views take the C interface as a template parameter, `iconv_functions`, rather than calling `::iconv` directly, so that a program can supply a different implementation of the same interface, whether another library's or a test's. `make_real_iconv_fns` is the one bound to the platform's.

0.13 Wording

These clauses are generated, as the abstract says, and transcluded into this paper by its build. The generator, the marked-up headers and the drift check are all in the repository cited in References; a reviewer can regenerate this section and diff it.

Wording is relative to the current working draft.

0.13.1 Editing instructions

Add the headers `<transcode>` and `<null_term>` to the table of C++ library headers in [headers].

Add the following to the normative references in [intro.refs]:

— WHATWG, *Encoding Standard*, Review Draft, June 2025, available at https://encoding.spec.whatwg.org/review-drafts/2025-06/.

[*Note*: the Encoding Standard is a Living Standard, and a normative reference has to name a dated document. WHATWG publishes Review Drafts, dated snapshots of a standard, for that purpose, and the reference above is the most recent [WHATWG-Encoding-RD]. The reference implementation's codec tables and conformance vectors are generated from a later commit, `a985b62a9b45c17da3e17a9f0a0b4e30c34c4a8a` (21 May 2026) [WHATWG-Encoding]. The five commits between the two change links, repository files and the wording of one `TextDecoderStream` algorithm; the encodings, their labels and their index data are the same in both. — *end note*]

Add the following macro to [version.syn], in the place the table's alphabetical order puts it:

```
#define __cpp_lib_transcode_view 202XXXL // also in <transcode>, <null_term>
```

Add a new clause [transcode], "Text transcoding", as follows.

[*Editor's note*: the clause headings below carry their stable names. `[transcode.syn]` and `[null.term.syn]` are the two header synopses; every other clause specifies what one of them declares. — *end note*]

[*Editor's note*: the synopsis below is rendered without its nested namespace blocks. The declarations of `[transcode.whatwg.encoding]`, `[transcode.whatwg.label]`, `[transcode.whatwg.sniff]`, `[transcode.whatwg.decoders]`, `[transcode.whatwg.encoders]`, `[transcode.whatwg.decode]` and `[transcode.whatwg.encode]` are members of namespace `std::whatwg`, as is every other name the synopsis spells with the `whatwg::` prefix, including `whatwg::transcode` in `[transcode.pipeline]`, `whatwg::transcode_string` in `[transcode.string]` and the `whatwg::` forms of `decode_to`, `encode_to`, `decode_into` and `encode_into` in `[transcode.bulk]`. In `[null.term.syn]`, `null_term` is a member of namespace `std::views`. The remaining declarations are members of namespace `std`. — *end note*]

Header `<transcode>` synopsis [transcode.syn]

```
namespace std {
  // [transcode.errors], error types

  enum class iconv_error {
    invalid_sequence,
    incomplete_sequence,
    output_full,
```

```
  open_failed,
  system_error,
};

enum class error_mode {
  replacement,
  report,
};

// [transcode.errors], error types

enum class error {
  invalid_byte,
  truncated_sequence,
  overlong_encoding,
  surrogate_code_point,
  out_of_range,
  unmapped_codepoint,
};

// [transcode.reqs], range requirements

template<typename T>
concept legacy-byte-type =
    same_as<T, char> || same_as<T, signed char> || same_as<T, unsigned char> ||
    same_as<T, byte>; // exposition only

template<typename R>
concept legacy_byte_range = ranges::range<R> && !is_array_v<remove_cvref_t<R>> &&
                            legacy-byte-type<remove_cv_t<ranges::range_value_t<R>>>;

template<typename R>
concept unicode_scalar_range =
    ranges::input_range<R> && !is_array_v<remove_cvref_t<R>> &&
    same_as<remove_cv_t<ranges::range_value_t<R>>, char32_t>;

// [transcode.whatwg.encoding], encodings

enum class encoding {
  utf_8,
  replacement,
  x_user_defined,
  ibm866,
  iso_8859_2,
  iso_8859_3,
  iso_8859_4,
  iso_8859_5,
  iso_8859_6,
  iso_8859_7,
  iso_8859_8,
  iso_8859_8_i,
  iso_8859_10,
  iso_8859_13,
  iso_8859_14,
  iso_8859_15,
  iso_8859_16,
  koi8_r,
  koi8_u,
```

```
  macintosh,
  windows_874,
  windows_1250,
  windows_1251,
  windows_1252,
  windows_1253,
  windows_1254,
  windows_1255,
  windows_1256,
  windows_1257,
  windows_1258,
  x_mac_cyrillic,
  utf_16be,
  utf_16le,
  gbk,
  gb18030,
  big5,
  shift_jis,
  euc_jp,
  iso_2022_jp,
  euc_kr,
};

// [transcode.whatwg.label], label lookup

constexpr optional<encoding> get_encoding(string_view label) noexcept;

// [transcode.whatwg.label], label lookup

constexpr optional<encoding> get_encoding(text_encoding::id id) noexcept;

constexpr optional<text_encoding::id> to_text_encoding_id(encoding e) noexcept;

// [transcode.codec.reqs], codec requirements

struct decode_result {
  char32_t code_point{};
  whatwg::error error{};
  bool is_error{false};
  bool is_empty{false};
};

struct encode_result {
  unsigned char bytes[8]{};
  int count{0};
  bool is_error{false};
};

// [transcode.codec.reqs], codec requirements

template<typename C>
concept text_decoder = semiregular<C> && requires(C& c, const unsigned char*& iter,
                                                  const unsigned char* sentinel) {
  { c.decode_one(iter, sentinel) } -> same_as<decode_result>;
};

template<typename C>
concept single_byte_text_decoder =
```

```
    text_decoder<C> && requires(const C& c, unsigned char byte) {
      { c.decode_byte(byte) } -> same_as<char32_t>;
    };

template<typename C>
concept text_encoder = semiregular<C> && requires(C& c, char32_t cp) {
  { c.encode_one(cp) } -> same_as<encode_result>;
};

template<typename C>
concept single_byte_text_encoder =
    text_encoder<C> && requires(const C& c, char32_t scalar) {
      { c.encode_byte(scalar) } -> same_as<optional<unsigned char>>;
    };

template<typename C>
concept flushable_text_decoder = text_decoder<C> && requires(C& c) {
  { c.flush() } -> same_as<optional<decode_result>>;
};

template<typename C>
concept bom_aware_text_decoder =
    text_decoder<C> &&
    requires(C& c, const unsigned char*& iter, const unsigned char* sentinel) {
      { c.decode_first(iter, sentinel) } -> same_as<decode_result>;
    };

template<typename C>
concept flushable_text_encoder = text_encoder<C> && requires(C& c) {
  { c.flush() } -> same_as<optional<encode_result>>;
};

// [transcode.whatwg.decoders], decoders

template<encoding C>
class decoder {
  [[no_unique_address]] detail::decode_state_t<C> state{}; // exposition only

public:
  // [transcode.whatwg.decoders], decoding
  template<input_iterator I, sentinel_for<I> S>
  constexpr decode_result decode_one(I& current, S end);

  template<input_iterator I, sentinel_for<I> S>
  constexpr decode_result decode_first(I& current, S end)
    requires strips-leading-bom<C>;

  constexpr char32_t decode_byte(unsigned char byte) const
    requires random-access-decode-codec<C>;

  constexpr optional<decode_result> flush()
    requires has-decode-state<C>;

  friend constexpr bool operator==(const decoder&, const decoder&) = default;
};

// [transcode.whatwg.encoders], encoders
```

```
template<encoding C>
  requires whatwg-encode-codec<C>
class encoder {
  [[no_unique_address]] detail::encode_state_t<C> state{}; // exposition only

public:
  // [transcode.whatwg.encoders], encoding
  constexpr encode_result encode_one(char32_t scalar);

  constexpr optional<unsigned char> encode_byte(char32_t scalar) const
    requires random-access-encode-codec<C>;

  constexpr optional<encode_result> flush()
    requires(C == encoding::iso_2022_jp);

  friend constexpr bool operator==(const encoder&, const encoder&) = default;
};

// [transcode.whatwg.dynamic], codecs for an encoding chosen at run time

class dynamic_decoder {
  detail::decoder_variant decoder; // exposition only

public:
  // [transcode.whatwg.dynamic], construction
  constexpr dynamic_decoder() noexcept;
  constexpr explicit dynamic_decoder(whatwg::encoding e);

  constexpr whatwg::encoding encoding() const noexcept;

  // [transcode.whatwg.dynamic], decoding
  template<input_iterator I, sentinel_for<I> S>
  constexpr decode_result decode_one(I& current, S end);

  template<input_iterator I, sentinel_for<I> S>
  constexpr decode_result decode_first(I& current, S end);

  constexpr optional<decode_result> flush();

  friend constexpr bool operator==(const dynamic_decoder&,
                                   const dynamic_decoder&) = default;
};

class dynamic_encoder {
  detail::encoder_variant encoder; // exposition only

public:
  // [transcode.whatwg.dynamic], construction
  constexpr dynamic_encoder() noexcept;
  friend constexpr optional<dynamic_encoder> make_dynamic_encoder(whatwg::encoding e);

  constexpr whatwg::encoding encoding() const noexcept;

  // [transcode.whatwg.dynamic], encoding
  constexpr encode_result encode_one(char32_t scalar);

  constexpr optional<encode_result> flush();
```

```
  friend constexpr bool operator==(const dynamic_encoder&,
                                   const dynamic_encoder&) = default;
};

// [transcode.whatwg.dynamic], creating an encoder for an encoding chosen at run time

// [transcode.codec.decode], class template decode_view

template<text_decoder Codec, ranges::input_range R,
         error_mode E = error_mode::replacement>
  requires legacy_byte_range<R>
class decode_view : public ranges::view_interface<decode_view<Codec, R, E>> {
  R base;                             // exposition only
  [[no_unique_address]] Codec codec; // exposition only

  class iterator; // exposition only

public:
  // [transcode.codec.decode], construction and access
  constexpr explicit decode_view(R base, Codec codec = {});

  constexpr const R& base() const& noexcept;
  constexpr R base() &&;

  constexpr iterator begin();
  constexpr iterator begin() const
    requires const-iterator-compatible-range<R> && const-sentinel-compatible-range<R>;
  constexpr iterator end()
    requires ranges::forward_range<R> && ranges::common_range<R>;
  constexpr iterator end() const
    requires ranges::forward_range<const R> && ranges::common_range<const R> &&
             const-iterator-compatible-range<R> && const-sentinel-compatible-range<R>;
  constexpr default_sentinel_t end() const;
};

template<text_decoder Codec> constexpr auto decode(Codec codec = {});

template<text_decoder Codec> constexpr auto decode_or_error(Codec codec = {});

namespace ranges {
  template<text_decoder Codec, input_range R, error_mode E>
    requires legacy_byte_range<R>
  inline constexpr bool enable_borrowed_range<decode_view<Codec, R, E>> =
      borrowed_range<R>;
}

// [transcode.codec.encode], class template encode_view

template<text_encoder Codec, ranges::input_range R,
         error_mode E = error_mode::replacement>
  requires unicode_scalar_range<R>
class encode_view : public ranges::view_interface<encode_view<Codec, R, E>> {
  R base;                             // exposition only
  [[no_unique_address]] Codec codec; // exposition only

  class iterator; // exposition only

public:
```

```
  // [transcode.codec.encode], construction and access
  constexpr explicit encode_view(R base, Codec codec = {});

  constexpr const R& base() const& noexcept;
  constexpr R base() &&;

  constexpr iterator begin();
  constexpr iterator begin() const
    requires const-iterator-compatible-range<R> && const-sentinel-compatible-range<R>;
  constexpr iterator end()
    requires ranges::forward_range<R> && ranges::common_range<R>;
  constexpr iterator end() const
    requires ranges::forward_range<const R> && ranges::common_range<const R> &&
             const-iterator-compatible-range<R> && const-sentinel-compatible-range<R>;
  constexpr default_sentinel_t end() const;
};

template<text_encoder Codec> constexpr auto encode(Codec codec = {});

template<text_encoder Codec> constexpr auto encode_or_error(Codec codec = {});

namespace ranges {
  template<text_encoder Codec, input_range R, error_mode E>
    requires unicode_scalar_range<R>
  inline constexpr bool enable_borrowed_range<encode_view<Codec, R, E>> =
      borrowed_range<R>;
}

// [transcode.whatwg.decode], decoding adaptors

template<whatwg::encoding C> inline constexpr unspecified decode;

template<whatwg::encoding C> inline constexpr unspecified decode_or_error;

// [transcode.whatwg.encode], encoding adaptors

template<whatwg::encoding C> inline constexpr unspecified encode;

template<whatwg::encoding C> inline constexpr unspecified encode_or_error;

// [transcode.pipeline], transcoding pipelines

template<text_decoder From, text_encoder To>
constexpr auto transcode(From from, To to);

// [transcode.pipeline], transcoding pipelines

template<encoding From, encoding To> inline constexpr unspecified transcode;

// [transcode.string], eager transcoding

constexpr bool has_encoder(whatwg::encoding encoding) noexcept;

optional<string> transcode_string(span<const char> src, whatwg::encoding from,
                                  whatwg::encoding to);

optional<string> transcode_string(span<const char> src, string_view from_label,
                                  string_view to_label);
```

```
// [transcode.bulk], eager bulk conversion

template<encoding C, legacy_byte_range R>
constexpr vector<char32_t> decode_to(R&& source);

template<encoding C, typename Container = string, unicode_scalar_range R>
constexpr Container encode_to(R&& source);

template<encoding C, legacy_byte_range R, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, Output output);

template<encoding C, unicode_scalar_range R, output_iterator<char> Output>
constexpr void encode_into(R&& source, Output output);

// [transcode.bulk], with the encoding chosen at run time

template<legacy_byte_range R>
constexpr vector<char32_t> decode_to(R&& source, encoding e);

template<legacy_byte_range R>
constexpr optional<vector<char32_t>> decode_to(R&& source, string_view label);

template<typename Container = string, unicode_scalar_range R>
constexpr optional<Container> encode_to(R&& source, encoding e);

template<typename Container = string, unicode_scalar_range R>
constexpr optional<Container> encode_to(R&& source, string_view label);

template<legacy_byte_range R, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, encoding e, Output output);

template<unicode_scalar_range R, output_iterator<char> Output>
constexpr bool encode_into(R&& source, encoding e, Output output);

// [transcode.bulk], or the first error

template<encoding C, legacy_byte_range R>
constexpr expected<vector<char32_t>, error> decode_to_or_error(R&& source);

template<encoding C, typename Container = string, unicode_scalar_range R>
constexpr expected<Container, error> encode_to_or_error(R&& source);

// [transcode.bulk], eager bulk conversion

template<legacy_byte_range R, text_decoder Codec>
constexpr vector<char32_t> decode_to(R&& source, Codec codec);

template<typename Container = string, unicode_scalar_range R, text_encoder Codec>
constexpr Container encode_to(R&& source, Codec codec);

template<legacy_byte_range R, text_decoder Codec, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, Codec codec, Output output);

template<unicode_scalar_range R, text_encoder Codec, output_iterator<char> Output>
constexpr void encode_into(R&& source, Codec codec, Output output);

// [transcode.bulk], or the first error
```

```
template<legacy_byte_range R, text_decoder Codec>
constexpr expected<vector<char32_t>, whatwg::error> decode_to_or_error(R&& source,
                                                                       Codec codec);

template<typename Container = string, unicode_scalar_range R, text_encoder Codec>
constexpr expected<Container, whatwg::error> encode_to_or_error(R&& source,
                                                                Codec codec);

// [transcode.whatwg.sniff], byte order mark sniffing

template<legacy_byte_range R>
  requires ranges::forward_range<R>
constexpr optional<encoding> bom_sniff(R&& r) noexcept;

// [transcode.iconv], iconv adaptors

inline constexpr size_t iconv_min_buffer_size = 4;

struct iconv_functions {
  iconv_t (*open)(const char* tocode, const char* fromcode);
  size_t (*convert)(iconv_t cd, char** inbuf, size_t* inbytesleft, char** outbuf,
                    size_t* outbytesleft);
  int (*close)(iconv_t cd);
};

template<typename IconvFns, ranges::input_range R>
  requires legacy_byte_range<R>
class iconv_transcode_view
    : public ranges::view_interface<iconv_transcode_view<IconvFns, R>> {
  R base;            // exposition only
  IconvFns fns;      // exposition only
  const char* from;  // exposition only
  const char* to;    // exposition only
  span<char> buffer; // exposition only

public:
  class iterator; // exposition only

  // [transcode.iconv], construction and access
  explicit iconv_transcode_view(R base, IconvFns fns, const char* from,
                                const char* to, span<char> buf);

  const R& base() const& noexcept;
  R base() &&;

  iterator begin();
  default_sentinel_t end() const;
};

template<typename IconvFns, ranges::input_range R>
  requires legacy_byte_range<R>
class iconv_transcode_or_error_view
    : public ranges::view_interface<iconv_transcode_or_error_view<IconvFns, R>> {
  R base;            // exposition only
  IconvFns fns;      // exposition only
  const char* from;  // exposition only
  const char* to;    // exposition only
```

```
    span<char> buffer; // exposition only

  public:
    class iterator; // exposition only

    // [transcode.iconv], construction and access
    explicit iconv_transcode_or_error_view(R base, IconvFns fns, const char* from,
                                           const char* to, span<char> buf);

    const R& base() const& noexcept;
    R base() &&;

    iterator begin();
    default_sentinel_t end() const;
  };

  inline iconv_functions make_real_iconv_fns() noexcept;

  inline auto iconv_transcode(const char* from, const char* to, span<char> buf);

  inline auto iconv_transcode_or_error(const char* from, const char* to,
                                       span<char> buf);

  // [transcode.iconv], eager conversion

  template<typename Container = string, typename IconvFns, legacy_byte_range R>
  Container iconv_transcode_to(R&& source, const char* from, const char* to,
                               IconvFns fns);

  template<typename Container = string, legacy_byte_range R>
  Container iconv_transcode_to(R&& source, const char* from, const char* to);

  template<typename IconvFns, legacy_byte_range R, output_iterator<char> Output>
  Output iconv_transcode_into(R&& source, const char* from, const char* to,
                              Output output, IconvFns fns);

  template<legacy_byte_range R, output_iterator<char> Output>
  Output iconv_transcode_into(R&& source, const char* from, const char* to,
                              Output output);

  template<typename Container = string, typename IconvFns, legacy_byte_range R>
  expected<Container, iconv_error> iconv_transcode_to_or_error(R&& source,
                                                               const char* from,
                                                               const char* to,
                                                               IconvFns fns);

  template<typename Container = string, legacy_byte_range R>
  expected<Container, iconv_error> iconv_transcode_to_or_error(R&& source,
                                                               const char* from,
                                                               const char* to);
}
```

Error types [transcode.errors]

```
enum class iconv_error {
  invalid_sequence,
  incomplete_sequence,
```

```
  output_full,
  open_failed,
  system_error,
};
```

1 *Remarks*: An `iconv` conversion that fails reports one of these values, which include the three failures POSIX `iconv` distinguishes and failures at the boundary of that model. The enumerators have the meanings in the following table.

Table 1: Enum class `iconv_error`

Constant	Meaning
`invalid_sequence`	the input is not valid in the source encoding, or has no representation in the destination encoding (`EILSEQ`).
`incomplete_sequence`	the input ends in the middle of a multibyte sequence (`EINVAL`).
`output_full`	the conversion has no room left to write its result (`E2BIG`).
`open_failed`	the conversion descriptor could not be opened; `iconv_open` uses `EINVAL` for an unsupported conversion pair, so this condition is kept distinct from an incomplete input sequence.
`system_error`	the conversion failed with an error POSIX does not specify for `iconv`.

```
enum class error_mode {
  replacement,
  report,
};
```

2 *Remarks*: A view's error mode says how it reports a failure of the codec it drives. The enumerators have the meanings in the following table.

Table 2: Enum class `error_mode`

Constant	Meaning
`replacement`	a failure to decode yields U+FFFD REPLACEMENT CHARACTER. Ill-formed UTF-32 input is first replaced with U+FFFD and then encoded normally; when an encoder cannot represent a scalar value, the encode view yields `'?'`. The view's value type is the codec's own.
`report`	the view's value type is `expected<T, whatwg::error>`, and a failure yields an `unexpected` holding the error that occurred.

```
enum class error {
  invalid_byte,
  truncated_sequence,
  overlong_encoding,
  surrogate_code_point,
  out_of_range,
  unmapped_codepoint,
};
```

[3] *Remarks*: An operation that fails to decode or to encode reports one of these values. The enumerators have the meanings in the following table.

Table 3: Enum class `whatwg::error`

Constant	Meaning
`invalid_byte`	the input holds a byte the encoding does not allow in that position.
`truncated_sequence`	the input ends in the middle of a sequence.
`overlong_encoding`	the sequence encodes a value that a shorter sequence also encodes.
`surrogate_code_point`	a byte sequence decodes to, or a UTF-32 input code unit holds, a surrogate code point, which is not a Unicode scalar value.
`out_of_range`	a byte sequence decodes to, or a UTF-32 input code unit holds, a value greater than the largest Unicode scalar value.
`unmapped_codepoint`	the encoding has no representation for the Unicode scalar value being encoded.

Range requirements [transcode.reqs]

```
template<typename R>
concept legacy_byte_range = ranges::range<R> && !is_array_v<remove_cvref_t<R>> &&
                            legacy-byte-type<remove_cv_t<ranges::range_value_t<R>>>;
```

[1] *Remarks*: An array type does not satisfy `legacy_byte_range`, so that the terminating null character of a string literal or of a character array is not transcoded along with the rest of the array. `views::null_term` adapts a null-terminated character sequence, and `span` a counted buffer.

```
template<typename R>
concept unicode_scalar_range =
    ranges::input_range<R> && !is_array_v<remove_cvref_t<R>> &&
    same_as<remove_cv_t<ranges::range_value_t<R>>, char32_t>;
```

[2] *Remarks*: `unicode_scalar_range` constrains the type of a range and not its values. Its `char32_t` elements are interpreted as UTF-32 code units by an encode operation: a code unit that is not a Unicode scalar value is replaced with U+FFFD or reported as an error, according to the operation's error mode.

Codec requirements [transcode.codec.reqs]

[1] *Remarks*: What a codec's `decode_one` returns. `is_error` says whether the decode failed. When it did, `error` says why and `code_point` is U+FFFD REPLACEMENT CHARACTER; when it did not, `code_point` is the decoded Unicode scalar value and `error` is unspecified. `is_empty` says the call consumed input but produced nothing – an escape sequence that only changes the decoder's mode, a byte order mark with nothing after it: when it is `true`, `code_point`, `error` and `is_error` are unspecified and the view asks again, or, if the input is exhausted, finishes.

[2] *Remarks*: What a codec's `encode_one` returns. `is_error` says whether the encode failed – the encoding has no representation for that scalar value. When it did not, `count` is the number of bytes written and `bytes[0]` through `bytes[count - 1]` are the encoded form. When it did, the view reports the error in report mode. In replacement mode it yields `bytes[0]` through `bytes[count - 1]` as the replacement if `count` is greater than zero, as an encoding does that must return to its initial state before the replacement, and `'?'` if `count` is zero.

```
template<typename C>
concept text_decoder = semiregular<C> && requires(C& c, const unsigned char*& iter,
                                                  const unsigned char* sentinel) {
```

```
  { c.decode_one(iter, sentinel) } -> same_as<decode_result>;
};
```

[3] *Remarks*: A type models `text_decoder` if it can decode one Unicode scalar value from a byte sequence. `c.decode_one(iter, sentinel)` reads from `iter` up to but not past `sentinel`, advances `iter` by the bytes it consumed, and returns what it decoded or the error that stopped it. It is called only with `iter != sentinel`. A codec that consumes no byte does not terminate the view that drives it.

[4] A codec that decodes a sequence with state keeps that state in itself, not in the range.

```
template<typename C>
concept single_byte_text_decoder =
    text_decoder<C> && requires(const C& c, unsigned char byte) {
      { c.decode_byte(byte) } -> same_as<char32_t>;
    };
```

[5] *Remarks*: A type models `single_byte_text_decoder` if each byte decodes to one scalar value independently of every other: `c.decode_byte(byte)` returns what `byte` alone decodes to, and U+FFFD if it decodes to nothing. For byte values `0x80` and above, U+FFFD is reserved as that "decodes to nothing" signal; the author of such a codec cannot map one of those bytes to U+FFFD itself. A view over such a codec is a `random_access_range`, since the nth element is the nth byte.

```
template<typename C>
concept text_encoder = semiregular<C> && requires(C& c, char32_t cp) {
  { c.encode_one(cp) } -> same_as<encode_result>;
};
```

[6] *Remarks*: A type models `text_encoder` if it can encode one Unicode scalar value to bytes. `c.encode_one(cp)` returns the encoded form of `cp`, or an error if the encoding has no representation for it. An encode view validates its UTF-32 input before calling this function, so `cp` is always a Unicode scalar value.

```
template<typename C>
concept single_byte_text_encoder =
    text_encoder<C> && requires(const C& c, char32_t scalar) {
      { c.encode_byte(scalar) } -> same_as<optional<unsigned char>>;
    };
```

[7] *Remarks*: A type models `single_byte_text_encoder` if each Unicode scalar value encodes to exactly one byte, independently of every other, or to nothing: `c.encode_byte(scalar)` returns the byte `scalar` alone encodes to, or `nullopt` if the encoding has no representation for it. Its `encode_one` agrees with `encode_byte`, never produces more than one byte, and carries no state, so a type that also models `flushable_text_encoder` must not model this concept. A view over such an encoder is a `random_access_range`, since the nth byte is the encoding of the nth scalar value.

```
template<typename C>
concept flushable_text_decoder = text_decoder<C> && requires(C& c) {
  { c.flush() } -> same_as<optional<decode_result>>;
};
```

[8] *Remarks*: A type models `flushable_text_decoder` if it can owe output after the last byte. `c.flush()` is called when the input is exhausted, and again after each engaged result, until it returns `nullopt`. It returns what the codec still owes – a truncation error for a sequence that ended mid-way, say – or `nullopt` if it owes nothing. A codec that does not model this concept is asked for nothing at the end of the input.

```
template<typename C>
concept bom_aware_text_decoder =
    text_decoder<C> &&
    requires(C& c, const unsigned char*& iter, const unsigned char* sentinel) {
      { c.decode_first(iter, sentinel) } -> same_as<decode_result>;
    };
```

[9] *Remarks*: A type models `bom_aware_text_decoder` if it decodes the start of a text differently from the rest, as a decoder that discards a leading byte order mark does. `c.decode_first(iter, sentinel)` is called

instead of `decode_one` for the first element of a non-empty input, once, and has the same contract. A codec that does not model this concept has `decode_one` called for every element.

```
template<typename C>
concept flushable_text_encoder = text_encoder<C> && requires(C& c) {
  { c.flush() } -> same_as<optional<encode_result>>;
};
```

10 *Remarks*: A type models `flushable_text_encoder` if it can owe output after the last scalar value, as an encoding with shift states must return to its initial state. `c.flush()` is called when the input is exhausted, and again after each engaged result, until it returns `nullopt`. It returns the bytes the codec still owes, or `nullopt` if it owes nothing. A codec that does not model this concept is asked for nothing at the end of the input. An engaged result with `count == 0` and `is_error` false owes nothing and ends the view. An error result's bytes are used as an `encode_one` error's are.

Class template `decode_view` [transcode.codec.decode]

```
constexpr const R& base() const& noexcept;
```

1 *Returns*: *`base`*.

```
constexpr R base() &&;
```

2 *Returns*: `move(`*`base`*`)`.

3 *Remarks*: `decode_view<Codec, R, E>` presents the bytes of `R` as the Unicode scalar values `Codec` decodes them to, and decodes lazily. When `E` is `error_mode::replacement`, its value type is `char32_t` and a decoding error yields U+FFFD. When `E` is `error_mode::report`, its value type is `expected<char32_t, whatwg::error>` and a decoding error yields an element holding the error. The view does not validate a successful `code_point`; the program-supplied `Codec` is responsible for returning Unicode scalar values as required by `text_decoder`.

4 The view models `random_access_range` when `Codec` models `single_byte_text_decoder` and `R` models `random_access_range`. When it does, a byte whose value is `0x80` or above and for which `Codec::decode_byte` returns U+FFFD is reported as `whatwg::error::invalid_byte` in report mode and yields U+FFFD in replacement mode. This follows from the U+FFFD reservation required by `single_byte_text_decoder` ([transcode.codec.reqs]).

```
constexpr explicit decode_view(R base, Codec codec = {});
```

5 *Effects*: Initializes *`base`* with `std::move(base)` and *`codec`* with `codec`.

```
constexpr iterator begin();
```

6 *Returns*: An *`iterator`* over *`base`* and a copy of *`codec`*, positioned at its first decoded element.

```
constexpr iterator begin() const;
```

7 *Constraints*: `R` models *`const-iterator-compatible-range`* and `R` models *`const-sentinel-compatible-range`*.

8 *Returns*: An *`iterator`* over *`base`* and a copy of *`codec`*, positioned at its first decoded element.

```
constexpr iterator end();
```

9 *Constraints*: `R` models `forward_range` and `R` models `common_range`.

10 *Returns*: An *`iterator`* that compares equal to an iterator that has decoded every element of *`base`*.

```
constexpr iterator end() const;
```

11 *Constraints*:

(11.1) — `const R` models `forward_range`,
(11.2) — `const R` models `common_range`,
(11.3) — `R` models *`const-iterator-compatible-range`*,
(11.4) — `R` models *`const-sentinel-compatible-range`*.

12 *Returns*: An *iterator* that compares equal to an iterator that has decoded every element of *base*.

```
constexpr default_sentinel_t end() const;
```

13 *Returns*: `default_sentinel`.

```
template<text_decoder Codec> constexpr see below decode(Codec codec = {});
```

14 *Returns*: A range adaptor object over `codec`. Given a subexpression `E` that models `legacy_byte_range`, `decode(codec)(E)` and `E | decode(codec)` are each expression-equivalent to a `decode_view<Codec, views::all_t<decl` over `E` and a copy of `codec`. An `E` of array type is ill-formed.

```
template<text_decoder Codec> constexpr see below decode_or_error(Codec codec = {});
```

15 *Returns*: `decode(codec)` with `error_mode::report`: the view it adapts to has value type `expected<char32_t, whatwg::e` and a decoding error is the error rather than U+FFFD.

Class `decode_view::iterator` [transcode.codec.decode.iterator]

```
constexpr iterator(base_iter current, base_sent end, Codec codec);
```

16 *Effects*: Initializes *current* with `std::move(current)`, *end* with `std::move(end)` and the iterator's own copy of the codec with `codec`, then decodes the first element. When `Codec` models `bom_aware_text_decoder` and `current != end`, that element is decoded by `decode_first`, and, if its result has `is_empty`, decoding goes on as `operator++` does.

```
constexpr result_t operator*() const;
```

17 *Returns*: *value*.

```
constexpr iterator& operator++();
```

18 *Effects*: Decodes the next element of the base range, or, if none remains, makes `*this` equal to `end()`. A `decode_one` result with `is_empty` yields no element and decoding goes on. When `Codec` models `flushable_text_decoder`, exhausting the base range calls `flush()`, and calls it again after each engaged result, yielding what it returns until it returns `nullopt` or a result with `is_empty`.

19 *Returns*: `*this`.

```
constexpr iterator operator++(int);
```

20 *Constraints*: `R` models `forward_range`.

21 *Effects*: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr void operator++(int);
```

22 *Constraints*: `ranges::forward_range<R>` is `false`.

23 *Effects*: Equivalent to:

```
++*this;
```

Class template `encode_view` [transcode.codec.encode]

```
constexpr const R& base() const& noexcept;
```

1 *Returns*: *base*.

```
constexpr R base() &&;
```

2 *Returns*: `move(base)`.

3 *Remarks*: `encode_view<Codec, R, E>` validates the `char32_t` elements of `R` as UTF-32, presents the resulting scalar values as the bytes `Codec` encodes them to, and encodes lazily. When `E` is `error_mode::replacement`, its value type is `char`. When `E` is `error_mode::report`, its value type is `expected<char, whatwg::error>` and an error yields an element holding the error. Ill-formed UTF-32 is replaced with U+FFFD before `Codec::encode_one` is called in replacement mode and is reported without calling the codec in report mode. A custom codec therefore receives only Unicode scalar values. When `Codec::encode_one` fails and supplies bytes, the view in replacement mode yields those bytes as the replacement; if it supplies none, the replacement is `'?'`. In report mode the bytes are ignored.

4 The view models `random_access_range` when `Codec` models `single_byte_text_encoder` and `R` models `random_access_range`. When it does, a scalar value for which `Codec::encode_byte` returns `nullopt` is reported as `whatwg::error::unmapped_codepoint` in report mode and yields `'?'` in replacement mode.

```
constexpr explicit encode_view(R base, Codec codec = {});
```

5 *Effects*: Initializes *`base`* with `std::move(base)` and *`codec`* with `codec`.

```
constexpr iterator begin();
```

6 *Returns*: An *`iterator`* over *`base`* and a copy of *`codec`*, positioned at the first byte of its first encoded element.

```
constexpr iterator begin() const;
```

7 *Constraints*: `R` models *`const-iterator-compatible-range`* and `R` models *`const-sentinel-compatible-range`*.

8 *Returns*: An *`iterator`* over *`base`* and a copy of *`codec`*, positioned at the first byte of its first encoded element.

```
constexpr iterator end();
```

9 *Constraints*: `R` models `forward_range` and `R` models `common_range`.

10 *Returns*: An *`iterator`* that compares equal to an iterator that has encoded every element of *`base`*.

```
constexpr iterator end() const;
```

11 *Constraints*:

(11.1) — `const R` models `forward_range`,
(11.2) — `const R` models `common_range`,
(11.3) — `R` models *`const-iterator-compatible-range`*,
(11.4) — `R` models *`const-sentinel-compatible-range`*.

12 *Returns*: An *`iterator`* that compares equal to an iterator that has encoded every element of *`base`*.

```
constexpr default_sentinel_t end() const;
```

13 *Returns*: `default_sentinel`.

```
template<text_encoder Codec> constexpr see below encode(Codec codec = {});
```

14 *Returns*: A range adaptor object over `codec`. Given a subexpression `E` that models `unicode_scalar_range`, `encode(codec)(E)` and `E | encode(codec)` are each expression-equivalent to an `encode_view<Codec, views::all_t<dec` over `E` and a copy of `codec`.

```
template<text_encoder Codec> constexpr see below encode_or_error(Codec codec = {});
```

15 *Returns*: `encode(codec)` with `error_mode::report`: the view it adapts to has value type `expected<char, whatwg::error` and an encoding or UTF-32 validation error is reported rather than replaced.

Class `encode_view::iterator` [transcode.codec.encode.iterator]

```
constexpr iterator(base_iter current, base_sent end, Codec codec);
```

16 *Effects*: Initializes *`current`* with `std::move(current)`, *`end`* with `std::move(end)` and the iterator's own copy of the codec with `codec`, then encodes the first element.

```
constexpr result_t operator*() const;
```

17 *Returns*: The byte at the current position of the encoded element, or the error the encoding of that element produced.

```
constexpr iterator& operator++();
```

18 *Effects*: Advances to the next byte of the encoded element, encoding the next element of the base range when the current one is exhausted; if no element remains, makes `*this` equal to `end()`. When `Codec` models `flushable_text_encoder`, exhausting the base range calls `flush()`, and calls it again after each engaged result, yielding the bytes it returns until it returns `nullopt`. An engaged result with `count == 0` whose `is_error` is false owes nothing and ends the view. An error result's bytes are used as an `encode_one` error's are: in replacement mode a non-empty `bytes[0, count)` is yielded as the replacement and otherwise `'?'` is; in report mode one `unexpected` element is yielded either way.

19 *Returns*: `*this`.

```
constexpr iterator operator++(int);
```

20 *Constraints*: `R` models `forward_range`.

```
constexpr void operator++(int);
```

21 *Constraints*: `ranges::forward_range<R>` is `false`.

Transcoding pipelines [transcode.pipeline]

```
template<text_decoder From, text_encoder To>
constexpr see below transcode(From from, To to);
```

1 *Returns*: A range adaptor object over `from` and `to`. Given a subexpression `E` that models `legacy_byte_range`, `transcode(from, to)(E)` and `E | transcode(from, to)` are each expression-equivalent to `E | decode(from) | encode`

2 *Remarks*: `from` and `to` may be the library's codec objects (`whatwg::decoder`, `whatwg::encoder`) or a program's own, in any combination. `whatwg::transcode<From, To>` ([transcode.pipeline]) is defined in terms of this function, with codecs named by enumerators rather than supplied as values.

```
template<encoding From, encoding To> inline constexpr unspecified transcode;
```

3 *Remarks*: `whatwg::transcode<From, To>` is `transcode(whatwg::decoder<From>{}, whatwg::encoder<To>{})` ([transcode.pipeline]), and so is `E | whatwg::decode<From> | whatwg::encode<To>` for a subexpression `E` that models `legacy_byte_range`. An `E` of array type is ill-formed.

4 The composition is the specification: the pipeline decodes and re-encodes one element at a time, and it is lazy because its parts are. A scalar value `To` cannot represent is an encoding error, reported as `whatwg::encode` reports one; a byte sequence `From` does not allow is a decoding error, and the U+FFFD it decodes to is what gets encoded.

5 *Mandates:* The Encoding Standard defines an encoder for `To`.

Encodings [transcode.whatwg.encoding]

```
enum class encoding {
  utf_8,
  replacement,
  x_user_defined,
  ibm866,
  iso_8859_2,
  iso_8859_3,
  iso_8859_4,
  iso_8859_5,
  iso_8859_6,
  iso_8859_7,
```

```
  iso_8859_8,
  iso_8859_8_i,
  iso_8859_10,
  iso_8859_13,
  iso_8859_14,
  iso_8859_15,
  iso_8859_16,
  koi8_r,
  koi8_u,
  macintosh,
  windows_874,
  windows_1250,
  windows_1251,
  windows_1252,
  windows_1253,
  windows_1254,
  windows_1255,
  windows_1256,
  windows_1257,
  windows_1258,
  x_mac_cyrillic,
  utf_16be,
  utf_16le,
  gbk,
  gb18030,
  big5,
  shift_jis,
  euc_jp,
  iso_2022_jp,
  euc_kr,
};
```

[1] *Remarks*: Each enumerator denotes the encoding that the WHATWG Encoding Standard names correspondingly.

Label lookup [transcode.whatwg.label]

```
constexpr optional<encoding> get_encoding(string_view label) noexcept;
```

[2] *Returns*: The `encoding` whose WHATWG label set contains `label`, and `nullopt` if no encoding has that label. Leading and trailing ASCII whitespace is ignored and the comparison is ASCII case-insensitive, which is the Encoding Standard's "get an encoding" algorithm.

[3] *Remarks*: An encoding's labels are the alternative names a document may spell it with – `latin1`, `iso-8859-1` and `windows-1252` are all labels of `whatwg::encoding::windows_1252` – so a lookup that fails is a name no encoding claims, not an encoding that cannot be represented.

```
constexpr optional<encoding> get_encoding(text_encoding::id id) noexcept;
```

[4] *Returns*: `get_encoding(label)` ([transcode.whatwg.label]) for the first of the name and the aliases of `std::text_encoding(id)` that is a label of an encoding, and `nullopt` if there is none, which includes `id::other` and `id::unknown`.

[5] *Remarks*: This is the Encoding Standard's "get an encoding" applied to the registered names, so `std::text_encoding::id::ISOLatin1` is `encoding::windows_1252`.

```
constexpr optional<text_encoding::id> to_text_encoding_id(encoding e) noexcept;
```

[6] *Returns*: The `id` of the registered encoding whose name is the name of `e` in the Encoding Standard, and `nullopt` if that name has no registry entry, which includes `encoding::replacement`, `encoding::x_user_defined` and `encoding::x_mac_cyrillic`.

7 *Remarks*: IANA registers the Encoding Standard's names, so `encoding::windows_1252` is `std::text_encoding::id::wind` and not one of its aliases.

Byte order mark sniffing [transcode.whatwg.sniff]

```
template<legacy_byte_range R> constexpr optional<encoding> bom_sniff(R&& r) noexcept;
```

8 *Constraints*: `R` models `forward_range`.

9 *Returns*: The encoding `r` begins with a byte order mark for:

(9.1) — `whatwg::encoding::utf_8`, if `r` begins with `EF BB BF`;
(9.2) — `whatwg::encoding::utf_16be`, if `r` begins with `FE FF`;
(9.3) — `whatwg::encoding::utf_16le`, if `r` begins with `FF FE`;
(9.4) — `nullopt` otherwise.

10 *Remarks*: The mark is not consumed: `r` can be passed on unchanged.

Decoders [transcode.whatwg.decoders]

1 *Remarks*: `whatwg::decoder<C>` is the decoder, as a codec object, for the encoding `C` the Encoding Standard defines. It models `text_decoder` ([transcode.codec.reqs]) for every encoding, and the decode adaptor over it ([transcode.codec.decode]) yields what `whatwg::decode<C>` does. It models `single_byte_text_decoder` when `C` names an encoding that decodes one byte to one scalar value, `flushable_text_decoder` when `C` names an encoding whose decoding can leave a result owed after the last byte, and `bom_aware_text_decoder` when `C` names a UTF encoding: its `decode_first` discards one leading U+FEFF BYTE ORDER MARK from the text it decodes. *`strips-leading-bom`*`<C>` is `true` when `C` is `utf_8`, `utf_16be` or `utf_16le`, *`has-decode-state`*`<C>` is `true` when `C` is `utf_16be`, `utf_16le`, `gbk`, `gb18030`, `big5` or `iso_2022_jp`, and *`random-access-decode-codec`*`<C>` is `true` when `C` is `x_user_defined` or one of the Encoding Standard's single-byte encodings. It is equality comparable.

```
template<input_iterator I, sentinel_for<I> S>
constexpr decode_result decode_one(I& current, S end);
```

2 *Effects*: Decodes one scalar value from `[current, end)` as the encoding `C` says, advancing `current` past the bytes it consumed.

3 *Returns*: The scalar value or the error, or, for an escape sequence that produced no scalar value, a result with `is_empty`.

```
template<input_iterator I, sentinel_for<I> S>
constexpr decode_result decode_first(I& current, S end);
```

4 *Constraints*: `C` models *`strips-leading-bom`*.

5 *Effects*: Decodes one scalar value as `decode_one(current, end)` does. If that is U+FEFF, it is discarded and, unless the input is exhausted, the next scalar value decoded in its place.

6 *Returns*: The scalar value or the error, or, for a U+FEFF discarded with no input after it, a result with `is_empty`.

```
constexpr char32_t decode_byte(unsigned char byte) const;
```

7 *Constraints*: `C` models *`random-access-decode-codec`*.

8 *Returns*: `0xFFFD` if `byte` decodes to nothing in `C`, otherwise the scalar value `byte` decodes to.

```
constexpr optional<decode_result> flush();
```

9 *Constraints*: `C` models *`has-decode-state`*.

10 *Effects*: Decodes what the end of the input leaves owed, from an empty range.

11 *Returns*: `nullopt` if nothing is owed. Otherwise a scalar value still pending, or the error for a sequence the input ended in the middle of.

Encoders [transcode.whatwg.encoders]

1 *Remarks*: `whatwg::encoder<C>` is the encoder, as a codec object, for the encoding `C`. *`whatwg-encode-codec`*`<C>` is `true` for every encoding but `utf_16be`, `utf_16le` and `replacement`, for which the Encoding Standard defines no encoder, and *`random-access-encode-codec`*`<C>` is `true` when `C` is `x_user_defined` or one of the Encoding Standard's single-byte encodings. It models `text_encoder` ([transcode.codec.reqs]) and the encode adaptor over it ([transcode.codec.encode]) yields what `whatwg::encode<C>` does. It models `flushable_text_encoder` when `C` is `encoding::iso_2022_jp`, which owes an escape sequence returning to ASCII after the last scalar value. It models `single_byte_text_encoder` when *`random-access-encode-codec`*`<C>` is `true`. It is equality comparable.

```
constexpr encode_result encode_one(char32_t scalar);
```

2 *Effects*: Encodes `scalar`, which is a Unicode scalar value, as the encoding `C` says, updating the encoder's state if `C` has one.

3 *Returns*: The encoded bytes, or, if `C` has no representation for `scalar`, a result with `is_error`. An error from `iso_2022_jp` carries the bytes that replace `scalar`: an escape sequence returning to ASCII if the output was not in it, then `'?'`.

```
constexpr optional<unsigned char> encode_byte(char32_t scalar) const;
```

4 *Constraints*: `C` models *`random-access-encode-codec`*.

5 *Returns*: The byte `scalar` encodes to in the encoding `C`, or `nullopt` if `C` has no representation for it. Present for the single-byte encodings and `x_user_defined`.

```
constexpr optional<encode_result> flush();
```

6 *Constraints*: `C == encoding::iso_2022_jp` is `true`.

7 *Effects*: Returns the encoder to its initial state.

8 *Returns*: `nullopt` if the output is already in ASCII. Otherwise the escape sequence that returns it there.

Codecs for an encoding chosen at run time [transcode.whatwg.dynamic]

1 *Remarks*: `whatwg::dynamic_decoder` is a decoder, as a codec object, for an encoding chosen at run time. Each operation behaves as the same operation of `whatwg::decoder<E>` does, where `E` is the encoding it was constructed with. It models `text_decoder`, `flushable_text_decoder` and `bom_aware_text_decoder` ([transcode.codec.reqs]) whatever the encoding, so the decode adaptor over it ([transcode.codec.decode]) yields what `whatwg::decode<E>` does. It does not model `single_byte_text_decoder`. It is equality comparable; two objects are equal if they decode the same encoding and are in the same state.

```
constexpr dynamic_decoder() noexcept;
```

2 *Effects*: Constructs a decoder for `encoding::utf_8`.

```
constexpr explicit dynamic_decoder(whatwg::encoding e);
```

3 *Effects*: Constructs a decoder for `e`, in its initial state.

```
constexpr whatwg::encoding encoding() const noexcept;
```

4 *Returns*: The encoding this decoder decodes.

```
template<input_iterator I, sentinel_for<I> S>
constexpr decode_result decode_one(I& current, S end);
```

5 *Effects*: Equivalent to `d.decode_one(current, end)`, where `d` is the `whatwg::decoder<E>` this object behaves as.

```
template<input_iterator I, sentinel_for<I> S>
constexpr decode_result decode_first(I& current, S end);
```

6 *Effects*: Equivalent to `d.decode_first(current, end)` if the `whatwg::decoder<E>` this object behaves as has `decode_first`, and to `d.decode_one(current, end)` otherwise.

```
constexpr optional<decode_result> flush();
```

7 *Effects*: Equivalent to `d.flush()` if the `whatwg::decoder<E>` this object behaves as has `flush`.

8 *Returns*: The result of `d.flush()` if there is one, and `nullopt` otherwise.

9 *Remarks*: `whatwg::dynamic_encoder` is an encoder, as a codec object, for an encoding chosen at run time. Each operation behaves as the same operation of `whatwg::encoder<E>` does, where `E` is the encoding it was constructed with. It models `text_encoder` and `flushable_text_encoder` ([transcode.codec.reqs]) whatever the encoding, so the encode adaptor over it ([transcode.codec.encode]) yields what `whatwg::encode<E>` does. It does not model `single_byte_text_encoder`. It is equality comparable. An encoding with no encoder has no `dynamic_encoder`; `whatwg::make_dynamic_encoder` says which.

```
constexpr dynamic_encoder() noexcept;
```

10 *Effects*: Constructs an encoder for `encoding::utf_8`.

```
constexpr whatwg::encoding encoding() const noexcept;
```

11 *Returns*: The encoding this encoder encodes.

```
constexpr encode_result encode_one(char32_t scalar);
```

12 *Effects*: Equivalent to `e.encode_one(scalar)`, where `e` is the `whatwg::encoder<E>` this object behaves as.

```
constexpr optional<encode_result> flush();
```

13 *Effects*: Equivalent to `e.flush()` if the `whatwg::encoder<E>` this object behaves as has `flush`.

14 *Returns*: The result of `e.flush()` if there is one, and `nullopt` otherwise.

```
friend constexpr optional<dynamic_encoder> make_dynamic_encoder(whatwg::encoding e);
```

15 *Returns*: `nullopt` if `whatwg::has_encoder(e)` is `false` ([transcode.string]), and otherwise an encoder for `e`, in its initial state.

16 *Remarks*: This is the one check a program holding an encoding from its input makes, and it is made where the encoder is created.

Decoding views [transcode.whatwg.decode]

```
template<whatwg::encoding C> inline constexpr unspecified decode;
```

1 *Remarks*: `whatwg::decode<C>` is `decode(whatwg::decoder<C>{})` ([transcode.codec.decode]). It is the codec-object adaptor over the library's decoder for the encoding `C`, the decoder as the Encoding Standard defines it. The value type, the reporting of decoding errors, the laziness and the range properties of the view it adapts to are those of that adaptor. An argument of array type is ill-formed: a string literal carries a terminating null character that a decode would render, and `views::null_term` ([null.term.adaptor]) is how a null-terminated sequence becomes a range.

```
template<whatwg::encoding C> inline constexpr unspecified decode_or_error;
```

2 *Remarks*: `whatwg::decode_or_error<C>` is `decode_or_error(whatwg::decoder<C>{})` ([transcode.codec.decode]): it is `whatwg::decode<C>` with `error_mode::report`. The view it adapts to has value type `expected<char32_t, whatwg::e` and a decoding error is the error rather than U+FFFD.

Encoding views [transcode.whatwg.encode]

```
template<whatwg::encoding C> inline constexpr unspecified encode;
```

1 *Remarks*: `whatwg::encode<C>` is `encode(whatwg::encoder<C>{})` ([transcode.codec.encode]). It is the codec-object adaptor over the library's encoder for the encoding `C`, the encoder as the Encoding Standard defines it. The value type, the reporting of encoding errors, the laziness and the range properties of the

view it adapts to are those of that adaptor; the view is random access for the single-byte encodings, which `single_byte_text_encoder` ([transcode.codec.reqs]) provides. An argument of array type is ill-formed.

2 *Mandates:* The Encoding Standard defines an encoder for `C`. It defines none for `utf_16be`, `utf_16le` or `replacement`.

```
template<whatwg::encoding C> inline constexpr unspecified encode_or_error;
```

3 *Remarks*: `whatwg::encode_or_error<C>` is `encode_or_error(whatwg::encoder<C>{})` ([transcode.codec.encode]): it is `whatwg::encode<C>` with `error_mode::report`. The view it adapts to has value type `expected<char, whatwg::error` and an encoding or UTF-32 validation error is the error rather than a replacement.

4 *Mandates:* The Encoding Standard defines an encoder for `C`.

Eager transcoding [transcode.string]

```
constexpr bool has_encoder(whatwg::encoding encoding) noexcept;
```

1 *Returns*: `true` if the WHATWG Encoding Standard defines an encoder for `encoding`, and `false` otherwise.

```
optional<string> transcode_string(span<const char> src, whatwg::encoding from,
                                  whatwg::encoding to);
```

2 *Returns*: `nullopt` if the WHATWG Encoding Standard defines no encoder for `to`; otherwise, the bytes of `src`, decoded as `from` and re-encoded as `to`. Errors are substituted rather than reported: a byte sequence `from` does not allow decodes to U+FFFD, and a scalar value `to` cannot represent encodes to `'?'`.

3 *Remarks*: This is `transcode` ([transcode.pipeline]) run to completion into a `string`, which is what a caller who wants the whole result and not a view writes. `src | whatwg::transcode<From, To> | ranges::to<string>()` is the same thing with the codecs known at compile time.

```
optional<string> transcode_string(span<const char> src, string_view from_label,
                                  string_view to_label);
```

4 *Returns*: `whatwg::transcode_string(src, *whatwg::get_encoding(from_label), *whatwg::get_encoding(to_labe` and `nullopt` if either label names no encoding ([transcode.whatwg.label]) or the target encoding has no WHATWG encoder.

5 *Remarks*: This is the overload a program uses when the encodings are a runtime choice – a `Content-Type` header, a command-line option – which is what labels are for.

Eager bulk conversion [transcode.bulk]

```
template<encoding C, legacy_byte_range R>
constexpr vector<char32_t> decode_to(R&& source);
```

1 *Returns*: `forward<R>(source) | decode<C> | ranges::to<vector<char32_t>>()`.

2 *Remarks*: Each of these is the pipeline it names, run to completion. They add no behaviour to it: the errors are the decoding view's, substituted rather than reported ([transcode.whatwg.decode]), and a caller who wants them reported writes the pipeline with `whatwg::decode_or_error` instead.

```
template<encoding C, typename Container = string, unicode_scalar_range R>
constexpr Container encode_to(R&& source);
```

3 *Returns*: `forward<R>(source) | encode<C> | ranges::to<Container>()`.

4 *Remarks*: `Container` is the caller's choice of result, defaulted to `string`.

```
template<encoding C, legacy_byte_range R, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, Output output);
```

5 *Effects*: Equivalent to:

```
ranges::copy(forward<R>(source) | decode<C>, output);
```

```
template<encoding C, unicode_scalar_range R, output_iterator<char> Output>
constexpr void encode_into(R&& source, Output output);
```

6 *Effects*: Equivalent to:

```
ranges::copy(forward<R>(source) | encode<C>, output);
```

```
template<legacy_byte_range R>
constexpr vector<char32_t> decode_to(R&& source, encoding e);
```

7 *Returns*: `whatwg::decode_to<E>(std::forward<R>(source))`, where `E` is the enumerator equal to `e`.

8 *Remarks*: The encoding is examined once, not once per element.

```
template<legacy_byte_range R>
constexpr optional<vector<char32_t>> decode_to(R&& source, string_view label);
```

9 *Returns*: `nullopt` if `whatwg::get_encoding(label)` ([transcode.whatwg.label]) is `nullopt`, and otherwise `whatwg::decode_to(std::forward<R>(source), *whatwg::get_encoding(label))`.

```
template<typename Container = string, unicode_scalar_range R>
constexpr optional<Container> encode_to(R&& source, encoding e);
```

10 *Returns*: `nullopt` if the Encoding Standard defines no encoder for `e`, and otherwise `whatwg::encode_to<E, Container>(st` where `E` is the enumerator equal to `e`.

11 *Remarks*: The encoding is examined once, not once per element.

```
template<typename Container = string, unicode_scalar_range R>
constexpr optional<Container> encode_to(R&& source, string_view label);
```

12 *Returns*: `nullopt` if `whatwg::get_encoding(label)` ([transcode.whatwg.label]) is `nullopt`, and otherwise `whatwg::encode_to<Container>(std::forward<R>(source), *whatwg::get_encoding(label))`.

```
template<legacy_byte_range R, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, encoding e, Output output);
```

13 *Effects*: Equivalent to `whatwg::decode_into<E>(std::forward<R>(source), output)`, where `E` is the enumerator equal to `e`.

```
template<unicode_scalar_range R, output_iterator<char> Output>
constexpr bool encode_into(R&& source, encoding e, Output output);
```

14 *Effects*: If the Encoding Standard defines an encoder for `e`, equivalent to `whatwg::encode_into<E>(std::forward<R>(sour` where `E` is the enumerator equal to `e`; otherwise none.

15 *Returns*: Whether the Encoding Standard defines an encoder for `e`.

```
template<encoding C, legacy_byte_range R>
constexpr expected<vector<char32_t>, error> decode_to_or_error(R&& source);
```

16 *Returns*: `decode_to_or_error(std::forward<R>(source), whatwg::decoder<C>{})` ([transcode.bulk]).

17 *Remarks*: `whatwg::decode_to<C>(source)` with the errors reported rather than substituted: the whole text as scalar values, or the first error the Encoding Standard's decoder for `C` reports in it. For an encoding chosen at run time, `decode_to_or_error(source, whatwg::dynamic_decoder(e))` ([transcode.whatwg.dynamic]).

```
template<encoding C, typename Container = string, unicode_scalar_range R>
constexpr expected<Container, error> encode_to_or_error(R&& source);
```

18 *Returns*: `encode_to_or_error<Container>(std::forward<R>(source), whatwg::encoder<C>{})` ([transcode.bulk]).

19 *Remarks*: `whatwg::encode_to<C, Container>(source)` with the errors reported rather than substituted: the whole text as bytes, or the first scalar value that is not a Unicode scalar value or that `C` has no representation for.

20 *Mandates:* The Encoding Standard defines an encoder for `C`.

```
template<legacy_byte_range R, text_decoder Codec>
constexpr vector<char32_t> decode_to(R&& source, Codec codec);
```

21 *Returns*: `forward<R>(source) | decode(codec) | ranges::to<vector<char32_t>>()`.

22 *Remarks*: The same four operations as the `whatwg::` forms over a codec supplied as a value ([transcode.codec.reqs]), for a codec chosen at run time or one the `whatwg::encoding` enumeration does not name. As in every eager helper, the source comes first, then what to read or write it as, then any output.

```
template<typename Container = string, unicode_scalar_range R, text_encoder Codec>
constexpr Container encode_to(R&& source, Codec codec);
```

23 *Returns*: `forward<R>(source) | encode(codec) | ranges::to<Container>()`.

```
template<legacy_byte_range R, text_decoder Codec, output_iterator<char32_t> Output>
constexpr void decode_into(R&& source, Codec codec, Output output);
```

24 *Effects*: Equivalent to:

```
ranges::copy(forward<R>(source) | decode(codec), output);
```

```
template<unicode_scalar_range R, text_encoder Codec, output_iterator<char> Output>
constexpr void encode_into(R&& source, Codec codec, Output output);
```

25 *Effects*: Equivalent to:

```
ranges::copy(forward<R>(source) | encode(codec), output);
```

```
template<legacy_byte_range R, text_decoder Codec>
constexpr expected<vector<char32_t>, whatwg::error> decode_to_or_error(R&& source,
                                                                       Codec codec);
```

26 *Returns*: Let `v` be `std::forward<R>(source) | decode_or_error(codec)` ([transcode.codec.decode]). If an element of `v` holds an error, the error of the first such element; otherwise a `vector<char32_t>` holding the value of each element of `v`, in order.

27 *Remarks*: This is `decode_to(source, codec)` with the errors reported rather than substituted: the whole text, or the first thing wrong with it. The elements of `v` after the first error are not examined. A codec for an encoding chosen at run time is `whatwg::dynamic_decoder` ([transcode.whatwg.dynamic]).

```
template<typename Container = string, unicode_scalar_range R, text_encoder Codec>
constexpr expected<Container, whatwg::error> encode_to_or_error(R&& source,
                                                                Codec codec);
```

28 *Returns*: Let `v` be `std::forward<R>(source) | encode_or_error(codec)` ([transcode.codec.encode]). If an element of `v` holds an error, the error of the first such element; otherwise a `Container` holding the value of each element of `v`, in order.

29 *Remarks*: This is `encode_to<Container>(source, codec)` with the errors reported rather than substituted. The elements of `v` after the first error are not examined, and the encoder is not flushed past it.

iconv adaptors [transcode.iconv]

```
const R& base() const& noexcept;
```

1 *Returns*: *`base`*.

```
R base() &&;
```

[2] *Returns*: `move(base)`.

```
const R& base() const& noexcept;
```

[3] *Returns*: *base*.

```
R base() &&;
```

[4] *Returns*: `move(base)`.

```
inline constexpr size_t iconv_min_buffer_size = 4;
```

[5] *Remarks*: The smallest output buffer accepted by `iconv_transcode_view`. An `iconv` implementation can still report `E2BIG` without producing a byte when a destination encoding needs a larger indivisible output unit.

[6] *Remarks*: The three POSIX `iconv` entry points a view calls, as a value the program supplies. `open` opens a conversion descriptor, `convert` converts, and `close` closes it; each has the signature and the semantics POSIX gives the function of the same name.

[7] The facilities in this subclause are provided only when a POSIX-conforming `iconv` interface, including the required constants and typedefs, is available. The interface may be supplied by the operating system or by a third-party library. The type `iconv_t` is the type supplied by that interface and denotes its conversion descriptors.

[8] *Remarks*: `iconv_transcode_view<IconvFns, R>` presents the bytes of `R` converted from the encoding named by `from` to the encoding named by `to`, one element per output byte, using the `iconv` implementation `IconvFns` names. Conversion is lazy and proceeds in batches: the view converts into the caller's buffer, yields those bytes, and converts again.

[9] What the encoding names mean, which pairs convert, and what a conversion does with input the source encoding does not allow are determined by the supplied `iconv` interface, not this specification.

[10] A conversion failure is reported as `iconv_error` ([transcode.errors]).

[11] The output buffer is the caller's, and is not owned by the view. Its contents between two increments are unspecified, and the program must keep it alive for the lifetime of every iterator the view produces. If `iconv` reports `E2BIG` without producing a byte, the range ends because its `char` value type has no error channel. A program that needs to detect that condition uses `iconv_transcode_or_error`, which reports `iconv_error::output_full` instead.

```
explicit iconv_transcode_view(R base, IconvFns fns, const char* from, const char* to,
                              span<char> buf);
```

[12] *Preconditions*: `buf.size() >= iconv_min_buffer_size` is `true`.

[13] *Effects*: Initializes the view with `std::move(base)`, `std::move(fns)`, `from`, `to` and `buf`. No conversion descriptor is opened: `begin` opens one.

```
iterator begin();
```

[14] *Returns*: An *iterator* over *base* holding a conversion descriptor opened by *fns*`.open(`*to*`, `*from*`)`.

[15] *Remarks*: Each call opens a descriptor, so a view converts once per `begin`. The iterator owns the descriptor and closes it, which is what makes the adaptor leak-free where the POSIX interface is not.

```
default_sentinel_t end() const;
```

[16] *Returns*: `default_sentinel`.

[17] *Remarks*: `iconv_transcode_or_error_view<IconvFns, R>` is `iconv_transcode_view` ([transcode.iconv]) with the errors reported rather than skipped: its value type is `expected<char, iconv_error>`, and a conversion failure is an element holding the `iconv_error` POSIX reported rather than input the range passes over. The buffer has no minimum-size precondition. If it cannot hold one indivisible conversion or flush unit, the view reports `iconv_error::output_full`.

```
explicit iconv_transcode_or_error_view(R base, IconvFns fns, const char* from,
                                       const char* to, span<char> buf);
```

18 *Effects*: Initializes the view with `std::move(base)`, `std::move(fns)`, `from`, `to` and `buf`.

```
iterator begin();
```

19 *Returns*: An *iterator* over *base* holding a conversion descriptor opened by *fns*`.open(`*to*`, `*from*`)`.

```
default_sentinel_t end() const;
```

20 *Returns*: `default_sentinel`.

```
inline iconv_functions make_real_iconv_fns() noexcept;
```

21 *Returns*: An `iconv_functions` whose members are the `iconv_open`, `iconv` and `iconv_close` functions supplied by the available POSIX-conforming `iconv` interface.

```
inline see below iconv_transcode(const char* from, const char* to, span<char> buf);
```

22 *Preconditions*: `buf.size() >= iconv_min_buffer_size` is `true`.

23 *Returns*: A range adaptor object. Given a subexpression `E` that models `legacy_byte_range`, `iconv_transcode(from, to, buf)(E)` and `E | iconv_transcode(from, to, buf)` are each expression-equivalent to an `iconv_transcode_view` ([transcode.iconv]) over `E`, `make_real_iconv_fns()`, `from`, `to` and `buf`.

24 *Remarks*: `buf` is the caller's, and every iterator the adaptor produces converts into it. Two ranges adapted with the same buffer must not be iterated at the same time.

```
inline see below iconv_transcode_or_error(const char* from, const char* to,
                                          span<char> buf);
```

25 *Returns*: `iconv_transcode(from, to, buf)` with the errors reported rather than skipped: the view it adapts to has value type `expected<char, iconv_error>`.

26 *Remarks*: Unlike `iconv_transcode`, this adaptor accepts a buffer of any size. If `iconv` reports `E2BIG` without producing output, the adapted view reports `iconv_error::output_full`.

```
template<typename Container = string, typename IconvFns, legacy_byte_range R>
Container iconv_transcode_to(R&& source, const char* from, const char* to,
                             IconvFns fns);
```

27 *Returns*: A `Container` holding the bytes of `source` converted from `from` to `to` by `fns`. Input the conversion does not accept is skipped, as it is by `iconv_transcode_view` ([transcode.iconv]). The result is empty when the conversion descriptor cannot be opened – which is what `iconv_open` failing means, and is not distinguishable here from an empty input. An unexpected system failure ends conversion and returns the bytes produced before the failure. Use `iconv_transcode_to_or_error` when either distinction matters.

```
template<typename Container = string, legacy_byte_range R>
Container iconv_transcode_to(R&& source, const char* from, const char* to);
```

28 *Returns*: `iconv_transcode_to<Container>(forward<R>(source), from, to, make_real_iconv_fns())`.

```
template<typename IconvFns, legacy_byte_range R, output_iterator<char> Output>
Output iconv_transcode_into(R&& source, const char* from, const char* to, Output output,
                            IconvFns fns);
```

29 *Effects*: Converts the bytes of `source` from `from` to `to` by `fns` and writes them through `output`.

30 *Returns*: The value of `output` after the last byte written.

31 *Remarks*: If the conversion descriptor cannot be opened, no bytes are written and the unchanged iterator is returned. This is not distinguishable from empty input; use `iconv_transcode_to_or_error` when the distinction matters. An unexpected system failure similarly returns the iterator after the last byte successfully written.

```
template<legacy_byte_range R, output_iterator<char> Output>
Output iconv_transcode_into(R&& source, const char* from, const char* to,
                            Output output);
```

32 *Returns*: `iconv_transcode_into(forward<R>(source), from, to, move(output), make_real_iconv_fns())`.

```
template<typename Container = string, typename IconvFns, legacy_byte_range R>
expected<Container, iconv_error> iconv_transcode_to_or_error(R&& source,
                                                              const char* from,
                                                              const char* to,
                                                              IconvFns fns);
```

33 *Returns*: A `Container` holding the converted bytes, or the first `iconv_error` the conversion reported.

34 *Remarks*: This is the eager form of `iconv_transcode_or_error` ([transcode.iconv]): it stops at the first failure rather than skipping it, which is the difference between a conversion a program wants to know about and one it wants to get through.

```
template<typename Container = string, legacy_byte_range R>
expected<Container, iconv_error> iconv_transcode_to_or_error(R&& source,
                                                              const char* from,
                                                              const char* to);
```

35 *Returns*: `iconv_transcode_to_or_error<Container>(forward<R>(source), from, to, make_real_iconv_fns())`.

Classes `iconv_transcode_view::iterator` and `iconv_transcode_or_error_view::iterator` [transcode.iconv.iterator]

```
iterator(iterator&&) noexcept;
```

36 *Effects*: Takes over `other`'s conversion descriptor and leaves `other` holding none.

37 *Remarks*: The iterator is move-only. A conversion descriptor is an OS-owned handle that cannot be duplicated, so copying one would either close it twice or leak it.

```
iterator& operator=(iterator&&) noexcept;
```

38 *Effects*: Closes the descriptor `*this` holds, if any, then takes over `other`'s and leaves `other` holding none.

39 *Returns*: `*this`.

```
~iterator();
```

40 *Effects*: Closes the conversion descriptor, if `*this` holds one.

```
char operator*() const;
```

41 *Returns*: The converted byte at the current position of the output buffer.

```
iterator& operator++();
```

42 *Effects*: Advances to the next converted byte, converting more input when the buffer is exhausted; if no input and no unconverted bytes remain, makes `*this` equal to `default_sentinel`.

43 *Returns*: `*this`.

44 *Remarks*: Input the conversion does not accept is skipped one byte at a time – what POSIX reports as `EILSEQ` – so the range ends where the input does rather than at the first byte a converter refuses. A program that needs to know *that* it happened uses `iconv_transcode_or_error` ([transcode.iconv]).

```
void operator++(int);
```

45 *Effects*: Equivalent to:

```
++*this;
```

```
iterator(iterator&&) noexcept;
```

46 *Effects*: Takes over `other`'s conversion descriptor and leaves `other` holding none.

```
iterator& operator=(iterator&&) noexcept;
```

47 *Effects*: Closes the descriptor `*this` holds, if any, then takes over `other`'s and leaves `other` holding none.

48 *Returns*: `*this`.

```
~iterator();
```

49 *Effects*: Closes the conversion descriptor, if `*this` holds one.

```
expected<char, iconv_error> operator*() const;
```

50 *Returns*: The converted byte at the current position of the output buffer, or the `iconv_error` the conversion reported there.

```
iterator& operator++();
```

51 *Effects*: Advances to the next element, converting more input when the buffer is exhausted; if no input and no unconverted bytes remain, makes `*this` equal to `default_sentinel`.

52 *Returns*: `*this`.

```
void operator++(int);
```

53 *Effects*: Equivalent to:

```
++*this;
```

Header <null_term> synopsis [null.term.syn]

```
namespace std {
  struct null_sentinel_t {
    // [null.term.sentinel], comparison
    template<input_iterator I>
      requires requires(I i) {
        { *i == 0 };
      }
    friend constexpr bool operator==(const I& it, null_sentinel_t);
  };

  inline constexpr null_sentinel_t null_sentinel{};

  template<contiguous_iterator I>
  class null_term_view : public ranges::view_interface<null_term_view<I>> {
    I ptr{}; // exposition only

  public:
    // [null.term.view], construction and access
    constexpr null_term_view() = default;
    constexpr explicit null_term_view(I ptr);

    constexpr I begin() const;
    constexpr null_sentinel_t end() const;
  };

  // [null.term.adaptor], range adaptor object

  inline constexpr unspecified null_term;
}
```

Class null_sentinel_t [null.term.sentinel]

```
template<input_iterator I>
friend constexpr bool operator==(const I& it, null_sentinel_t);
```

1 *Constraints*: `requires(I i) { { *i == 0 }; }` is `true`.

2 *Returns*: `*it == 0`.

Class template `null_term_view` [null.term.view]

```
constexpr null_term_view() = default;
```

1 *Remarks*: The resulting view is singular. It may be assigned to and destroyed, but iterating it results in undefined behavior because the iterator returned by `begin()` has no reachable terminator.

```
constexpr explicit null_term_view(I ptr);
```

2 *Preconditions*: There is an iterator `i` reachable from `ptr` by a finite sequence of applications of `++` such that `*i == 0`, and every element from `ptr` through `i`, inclusive, is within the lifetime of a single object.

3 *Effects*: Initializes *`ptr`* with `ptr`.

```
constexpr I begin() const;
```

4 *Returns*: *`ptr`*.

```
constexpr null_sentinel_t end() const;
```

5 *Returns*: `null_sentinel_t()`.

`views::null_term` [null.term.adaptor]

```
inline constexpr unspecified null_term;
```

1 *Remarks*: The name `views::null_term` denotes a range adaptor object. Given a subexpression `E`, the expression `views::null_term(E)` is expression-equivalent to `null_term_view(E)` when `E` has pointer type, and to `null_term_view(static_cast<const T*>(E))` when `E` has array type with element type `T`. For any other type it is ill-formed, so that a range carrying no terminator is a diagnosed error and not a silent one. The precondition of `null_term_view`'s converting constructor applies to `E` after any array-to-pointer conversion.

0.14 Acknowledgements

— Zach Laine for `Boost.Text` and extensive work on Unicode in C++.
— Tom Honermann and SG16 for guidance on text encoding issues.
— The WHATWG for the Encoding Standard [WHATWG-Encoding].
— The Web Platform Tests project [WPT-Encoding] for comprehensive test vectors.

0.15 Prior Art

The design draws on several earlier libraries and proposals that explored encoding-aware iteration, transcoding APIs, and implementation techniques.

Tom Honermann's `text_view` library and paper [P0244R2],[text_view] established an early range-based model for enumerating encoded text as code points. It made the encoding explicit in the type system, and it ran into the ergonomics problem that follows from doing so: iteration has to surface decoding status somehow.

Zach Laine's `Boost.Text` [Boost.Text] integrates range-based text facilities with C++ algorithms and pipelines, and its Unicode-oriented design and normalization support mark out territory that a proposal about external byte streams should stay out of.

JeanHeyd Meneide's `P1629R1` proposal and `ztd.text` implementation [P1629R1],[ztd.text] go the other way: a broad text and transcoding library, eager and lazy interfaces both, encoding objects as values, extensible throughout. The surface here is narrower and fixed on browser-compatible WHATWG transcoding. But the split between lazy views and eager bulk helpers is the same admission that both usage styles are needed.

Henri Sivonen's `encoding_rs` [encoding_rs] is the important implementation prior art for WHATWG-compatible transcoding specifically. It targets the Encoding Standard directly, provides streaming and bulk-oriented APIs both, and validates against the web-platform tests while still chasing fast paths. Exact WHATWG semantics, WPT-derived conformance coverage, and eager helpers alongside the range adaptors are all lessons taken from it.

The deliberate departure from `text_view`, Boost.Text, and `ztd.text` is scope. Their general encoding abstractions can represent more models, but that breadth makes the common boundary operation harder to name and specify. This proposal fixes one compatibility model, WHATWG, and gives it direct decode, encode, and collection names. The narrower surface is the point, not an intermediate step toward another general text framework.

0.16 Relationship to Other Proposals

0.16.1 P2728 Unicode in the Library

[P2728R13] proposes transcoding views for UTF-8, UTF-16, and UTF-32 that operate on the strict Unicode character types (`char8_t`, `char16_t`, `char32_t`). This proposal overlaps in the UTF portions but differs in scope and philosophy:

Aspect	P2728	This Proposal
Input types	`char8_t`, `char16_t`, `char32_t`	`char`, `signed char`, `unsigned char`, `std::byte`
UTF-16 model	`char16_t` code units (native order)	Byte stream with explicit BE/LE
Encodings	UTF-8, UTF-16, UTF-32 only	40 `whatwg::encoding` enumerators, including WHATWG special and decode-only entries
Focus	Strict Unicode processing	Web/legacy interoperability
Use case	Internal Unicode manipulation	External data ingestion

The UTF-16 distinction is particularly important. P2728 operates on `char16_t`, which represents UTF-16 code units already in the platform's native byte order. This is appropriate for in-memory Unicode processing where the data originated from C++ code.

This proposal instead decodes UTF-16 from raw byte streams:

```
// P2728: char16_t input (native byte order, already code units)
std::u16string s = u"Hello";
auto codepoints = s | std::uc::to_utf32;

// This proposal: byte input with explicit byte order
std::vector<std::byte> network_data = read_socket();
auto codepoints = network_data | whatwg::decode<whatwg::encoding::utf_16be>;
```

When UTF-16 data arrives from external sources — network protocols, file formats, binary blobs — it arrives as bytes with a specific byte order, determined by the protocol or the BOM. The host's own endianness does not enter into it. The `whatwg::encoding::utf_16be` and `whatwg::encoding::utf_16le` variants decode these byte streams correctly either way.

The proposals are complementary:

— **P2728** is ideal when working with well-typed Unicode data within an application. Its strict typing catches encoding mismatches at compile time.

— **This proposal** is necessary when ingesting data from external sources — network protocols, file formats, databases — where bytes arrive as `char*` or `std::byte*`. The encoding is determined by metadata or heuristics that the C++ type system never sees.

A typical workflow might use this proposal to decode legacy-encoded input to `char32_t`, process it, then use P2728's facilities for UTF-to-UTF conversions within the application, and finally use this proposal again to encode output for a legacy system.

The UTF-8 decode/encode in this proposal follows WHATWG semantics (replacement character on error), which may differ slightly from P2728's error handling. Applications requiring strict UTF validation should prefer P2728 for that portion of the pipeline.

0.16.2 API Surface Comparison

The three implementation families in this proposal have matching API surfaces for every operation their encoding model supports. The P2728 column shows the parallel design in the proposed standard UTF transcoding views.

API	WHATWG	Pluggable codec	iconv	P2728
Codec identity	`whatwg::encoding_type` enum	`my_utf8_codec{}` type	`"UTF-8"` string	`char8_t`/`char16_t`/`char32_t`
Decode view	`whatwg::decode<C>(r)`	`decode(codec)`	`iconv_transcode_view(r,"UTF-8","UTF-32")`	`views::transcode_from_to<utf32>(r)`
Decode or-error view	`whatwg::decode_or_error<C>(r)`	`decode_or_error(codec)`	`iconv_transcode_view(r,"UTF-8","UTF-32",true)`	`views::transcode_from_to<utf32_or_error>(r)`
Encode view	`whatwg::encode<C>(r)`	`encode(codec)`	n/a [1]	`views::to_utf8` / `to_utf16`
Encode or-error view	`whatwg::encode_or_error<C>(r)`	`encode_or_error(codec)`	n/a [1]	`views::to_utf8_or_error`
Transcode pipeline	`whatwg::transcode_from_to(f,t)`	`transcode_from_to(f,t)`	`iconv_transcode(from,to,buf) |`	`v|views::transcode_from_to(f,t)`
Bulk decode → container	`whatwg::decode_to<C>(r)`	`decode_to(codec,r)`	n/a [1]	`v|ranges::to<>()`
Bulk encode → container	`whatwg::encode_to<C>(r)`	`encode_to(codec,r)`	n/a [1]	`v|ranges::to<>()`
Bulk transcode → container	n/a [2]	n/a [2]	`iconv_transcode_to(range,f,t)`	n/a [2]
Bulk decode → output iter	`whatwg::decode_into<C>(r,out)`	`decode_into(codec,r,out)`	n/a [1]	`ranges::copy(v, out)`
Bulk encode → output iter	`whatwg::encode_into<C>(r,out)`	`encode_into(codec,r,out)`	n/a [1]	`ranges::copy(v, out)`
Bulk transcode → output iter	n/a [2]	n/a [2]	`iconv_transcode_into(range,f,t,out)`	n/a [2]
Null-terminated input	`views::null_terminated(ptr)`	`views::null_term(ptr)`	`views::null_term(ptr)`	n/a [3]

API	WHATWG	Pluggable codec	iconv	P2728
Runtime label lookup	`whatwg::get_encoding("utf-8")`		n/a (string labels are the API)	n/a
Runtime transcode	`optional<string> whatwg::transcode_string(src, from, to)`		`iconv_transcode_to`	`r|f|t`
BOM sniffing	`whatwg::bom_sniff(range)`		n/a	n/a
Error type	`whatwg::error`	`whatwg::error`	`iconv_error`	`utf_transcoding_error`
Output element type	`char32_t`	`char32_t`	`char` (raw bytes)	`char32_t`
Input element type	`char`/`byte` (legacy)	`char`/`byte` (legacy)	`char` (any byte encoding)	`char8_t`/`char16_t`/`char32_t`
constexpr	yes	yes	no (POSIX userland library)	yes

Notes on n/a entries:

[1] **iconv is a byte byte transcoder.** It converts one byte encoding directly to another without exposing an intermediate `char32_t` stage. Encode (from `char32_t` to bytes) and decode (from bytes to `char32_t`) as separate steps are outside its model.

[2] **WHATWG and pluggable codecs compose decode and encode.** The named bulk helpers forward to `view | ranges::to<Container>()` and `ranges::copy(view, output)`, and a bulk transcode is `decode | encode` composed with `|` and collected the same way, so there is no separate compile-time-selected helper for it. `whatwg::transcode_string` is the eager transcoder, and it is in the *Runtime transcode* row below rather than this one because its codecs are values and not template parameters, which is the reason it exists. iconv performs single-pass byte→byte conversion and exposes it as a first-class bulk operation.

[3] **P2728 operates on typed Unicode character types** (`char8_t`, `char16_t`, `char32_t`); `views::null_term` produces a range of `char`. Bridging the two requires a reinterpret step that is outside both proposals.

Pluggable codecs are identified by C++ type. Codec selection happens at compile time through the type system; there is no runtime name-to-codec registry by design. Runtime transcode is similarly outside the model: you compose `decode(codec_a{}) | encode(codec_b{})` at compile time.

P2728 selects its codec from the character type. Selection is determined by the character types (`char8_t`, `char16_t`, `char32_t`), so runtime label lookup and runtime transcode are outside its model.

BOM sniffing is a property of the byte stream. `whatwg::bom_sniff()` examines the first bytes of a stream to detect UTF-8, UTF-16BE or UTF-16LE BOMs and returns the appropriate `whatwg::encoding` value. This is a WHATWG-specific facility; pluggable codecs and iconv operate on already-identified encodings.

P2728 operates on in-memory typed data where the encoding is known from the type. BOM detection is an I/O concern; a separate endian-converting view proposal addresses byte-order issues.

The shared design patterns across all four columns are the `_or_error` suffix for error-surfacing variants, `char32_t` as the interchange type, and composition with `|`.

0.17 References

— WHATWG Encoding Standard [WHATWG-Encoding]
— Web Platform Tests [WPT-Encoding]
— Reference implementation: https://github.com/steve-downey/transcode. The library is `beman.transcode`; it has not been published under the Beman project's organization yet, so that is where the code is.

[Boost.Text] Zach Laine. 2018. Boost.Text.
https://github.com/tzlaine/text
[D4322] Steve Downey. 2026. Algorithms for Trees.
https://github.com/bemanproject/typeclass/blob/main/papers/algorithms-for-trees.md
[encoding_rs] Henri Sivonen. 2018. encoding_rs.
https://github.com/hsivonen/encoding_rs
[P0244R2] Tom Honermann. 2017-06-13. Text_view: A C++ concepts and range based character encoding and code point enumeration library.
https://wg21.link/p0244r2
[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.
https://wg21.link/p1629r1
[P2728R13] Eddie Nolan. 2026-05-27. Unicode in the Library, Part 1: UTF Transcoding.
https://wg21.link/p2728r13
[P2871R3] Alisdair Meredith. 2023-12-18. Remove Deprecated Unicode Conversion Facets From C++26.
https://wg21.link/p2871r3
[P2873R3] Alisdair Meredith, Tom Honermann. 2025-03-17. Remove Deprecated locale category facets for Unicode from C++26.
https://wg21.link/p2873r3
[Schultke] Jan Schultke. 2026. Personal correspondence.
[Stream-Fusion] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. 2007. Stream Fusion: From Lists to Streams to Nothing at All. In *Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming (ICFP 2007)*, 2007. https://doi.org/10.1145/1291151.1291199
[text_view] Tom Honermann. 2017. text_view.
https://github.com/tahonermann/text_view
[WHATWG-Encoding] WHATWG. 2026-05-21. Encoding Standard (Living Standard, commit a985b62a9b45c17da3e17a9f0a0b4e30c34c4a8a, last updated 21 May 2026).
https://encoding.spec.whatwg.org/commit-snapshots/a985b62a9b45c17da3e17a9f0a0b4e30c34c4a8a/
[WHATWG-Encoding-RD] WHATWG. 2025-06. Encoding Standard (Review Draft, June 2025).
https://encoding.spec.whatwg.org/review-drafts/2025-06/
[WPT-Encoding] W3C Web Platform Tests Contributors. 2026. Web Platform Tests — Encoding.
https://github.com/web-platform-tests/wpt/tree/master/encoding
[ztd.text] JeanHeyd Meneide. 2021. ztd.text.
https://github.com/soasis/text